On-Device AI 응용

성균관대 신동군 교수

Outline

- LLM Pruning
- LLM Quantization
- Efficient Inference

Taylor Expansion Analysis on Pruning Error

- Evaluate pruning error induced by pruning synapses
  - The task of training neural network is to minimize the objective function.
  - The importance of a parameter can be quantified by the error induced by removing it (saliency 현저함).
  - The induced error (perturbation) can be approximated by a Taylor series

$$\delta L = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x};\ \mathbf{W}_P = \mathbf{W} - \delta\mathbf{W}) = \sum_i g_i \delta w_i + \frac{1}{2}\sum_{i,j} h_{ij}\delta w_i \delta w_j + O(\|\delta\mathbf{W}\|^3)$$

$$g_i = \frac{\partial L}{\partial w_i} \qquad h_{ij} = \frac{\partial^2 L}{\partial w_i \partial w_j}$$

Optimal Brain Damage [LeCun et al., NeurIPS 1989]

OBD: Second-Order-based Pruning

$$\delta L = \sum_i g_i \delta w_i + \frac{1}{2} \sum_{i,j} h_{ij} \delta w_i \delta w_j + O(\|\delta \mathbf{W}\|^3)$$

$$= \sum_i g_i \delta w_i + \frac{1}{2} \sum_i h_{ii} \delta w_i^2 + \frac{1}{2} \sum_{i \neq j} h_{ij} \delta w_i \delta w_j + O(\|\delta \mathbf{W}\|^3)$$

- Optimal Brain Damage assumes that
  - The neural network training has converged
    - first-order terms are neglected ($g_i = 0$)
  - The error caused by deleting each parameter is independent
    - cross terms are neglected ($h_{ij} = 0$)
  - The objective function is nearly quadratic: the last term is neglected
- The synapses with smaller induced error $|\delta L_i|$ will be removed

$$\delta L_i = L(\mathbf{x}; \mathbf{W}) - L(\mathbf{x}; \mathbf{W}_P | w_i = 0) \approx \frac{1}{2} h_{ii} w_i^2$$

Hessian Matrix *H* is difficult to compute.

Optimal Brain Damage [LeCun et al., NeurIPS 1989]

OBD: Second-Order-based Pruning

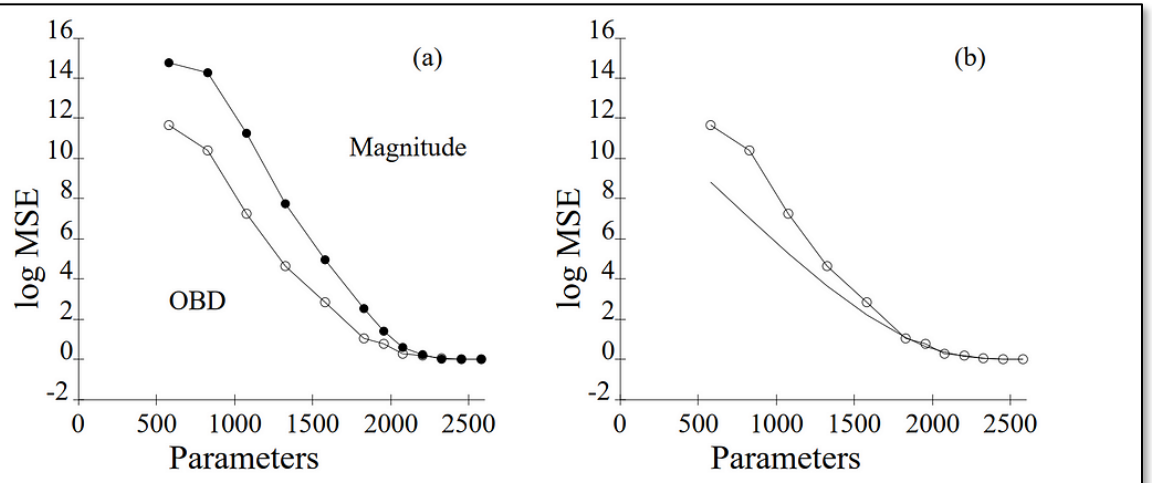


Figure 1: (a) Objective function (in dB) versus number of parameters for OBD (lower curve) and magnitude-based parameter deletion (upper curve). (b) Predicted and actual objective function versus number of parameters. The predicted value (lower curve) is the sum of the saliencies of the deleted parameters.

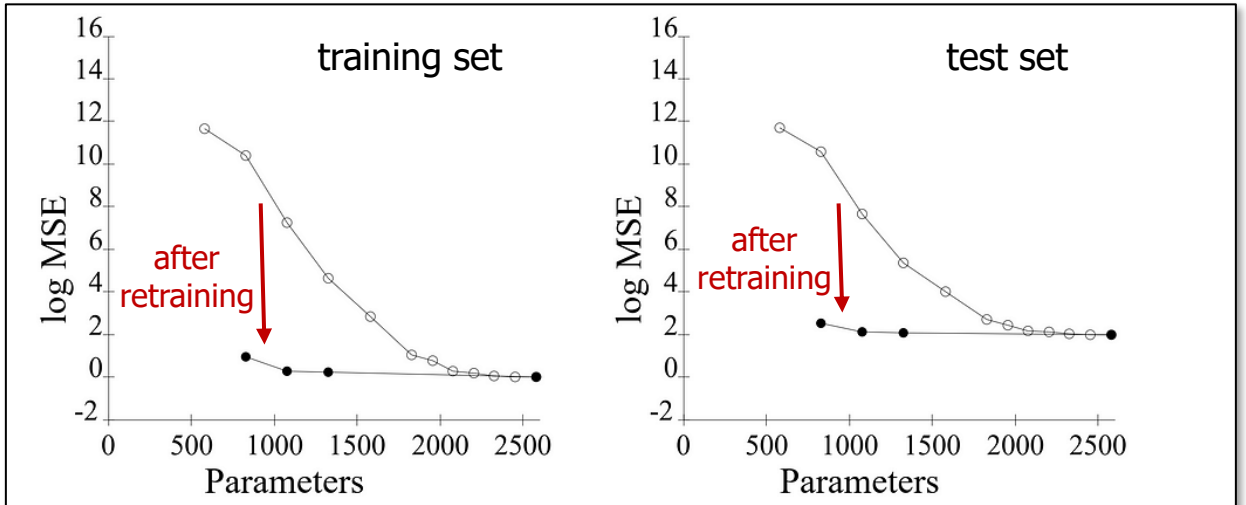


Figure 2: Objective function (in dB) versus number of parameters, without retraining (upper curve), and after retraining (lower curve). Curves are given for the training set (a) and the test set (b).

Optimal Brain Damage [LeCun et al., NeurIPS 1989]

Optimal Brain Surgeon (OBS)

- Consider the cross-terms within the Hessian matrix (remove the diagonality assumption of OBD).
- Determine optimal updates to the remaining weights based on the removal of a given parameter
- Simultaneously pruning and optimizing the model, no need for retraining.
- Optimal weight change and resulting change in error:
  - $\delta w = -\frac{w_q}{[H^{-1}]_{qq}} \cdot H^{-1}_{:,q}$ and $L_q = \frac{1}{2}\frac{w_q^2}{[H^{-1}]_{qq}}$

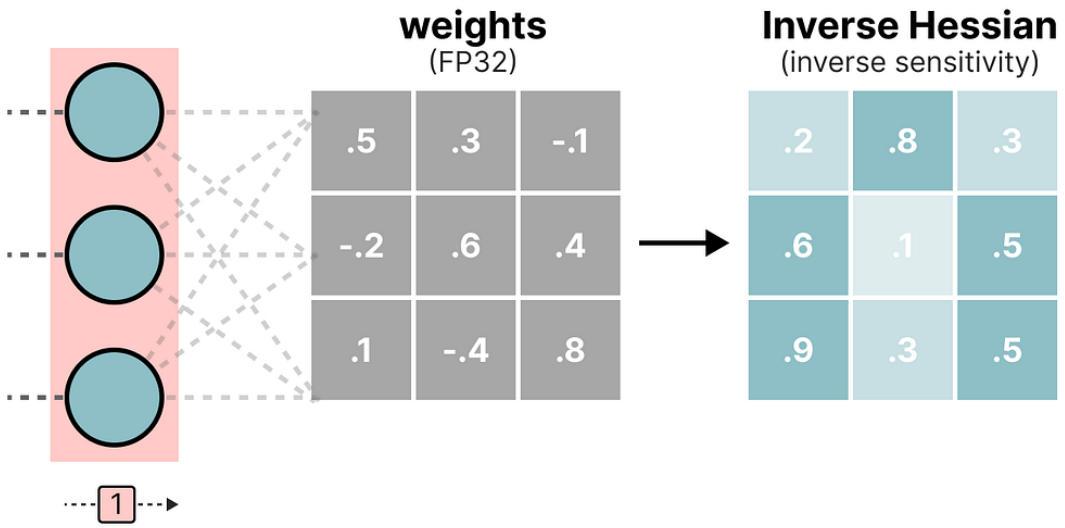
$[H^{-1}]_{qq}$: the qth diagonal entry of the inverse Hessian, and $H^{-1}_{:,q}$ is its qth column.

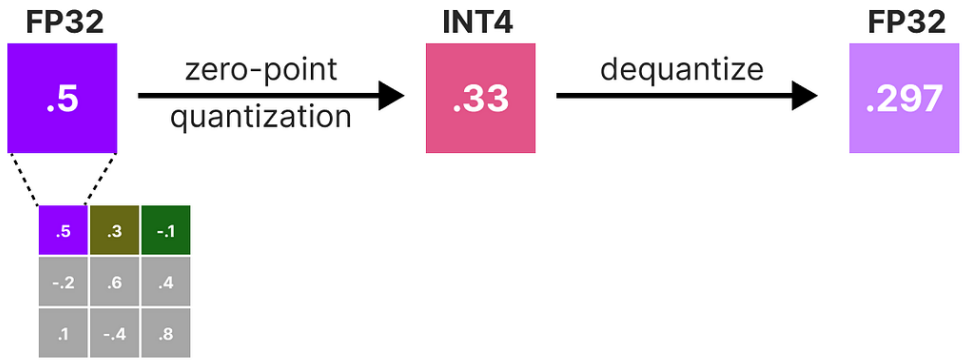
repeat
- Compute H^{-1}
- Find the q that gives the smallest L_q
- Update all weights δw

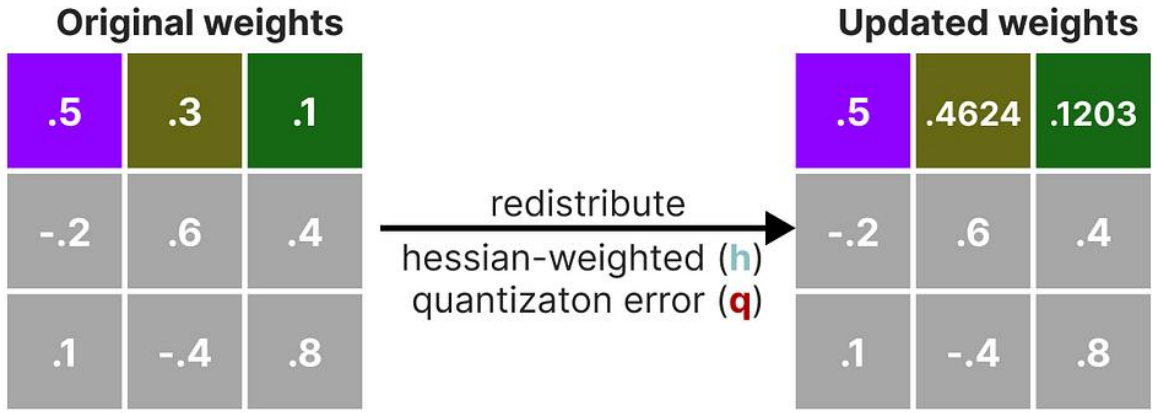
- Many follow-up methods for Hessian approximation (empirical Fisher)
  - WoodFisher (NeurIPS'20), M-FAC (NeurIPS'21), oBERT (EMNLP'22)

Second order derivatives for network pruning: Optimal Brain Surgeon [Hassibi et al., NeurIPS 1992]

Optimal Brain Surgeon (OBS)







$$q = \frac{x_1 - x_1}{h_1}$$ (hessian-weighted quantization error)

$$q = \frac{.5 - .297}{.2} = .203$$

$$x_2 = x_2 + q \cdot h_2$$ (update weight)

$$x_2 = .3 + .203 \cdot .8$$

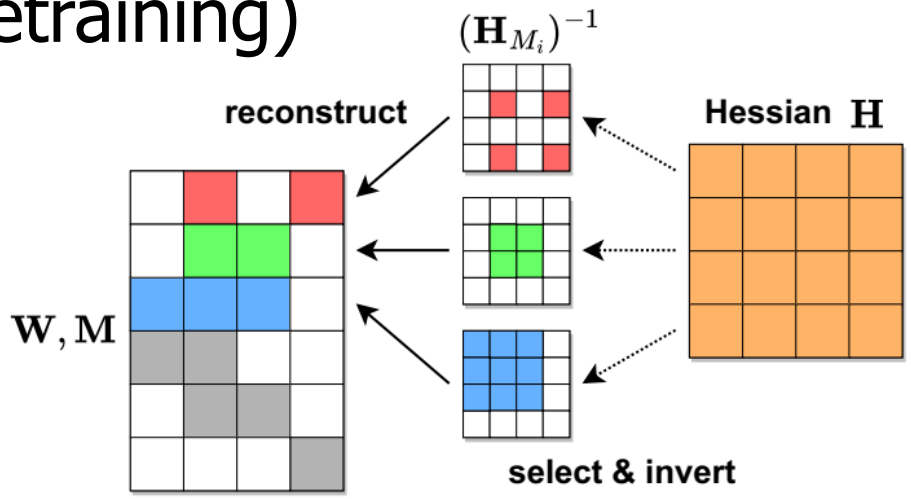
https://towardsdatascience.com/a-visual-guide-to-quantization-930ebcd9be94

LLM Pruning

- The enormous size of LLMs (100+ Billion Parameters) requires special considerations
- Magnitude Pruning preserves accuracy only until 10% sparsity.
- OBS takes hundreds of hours for 100+ billion parameters.
- Retraining to recover model performance after pruning is prohibitively expensive.

SparseGPT

- Unstructured & One-Shot pruning (No Retraining)
- Layer-wise approach

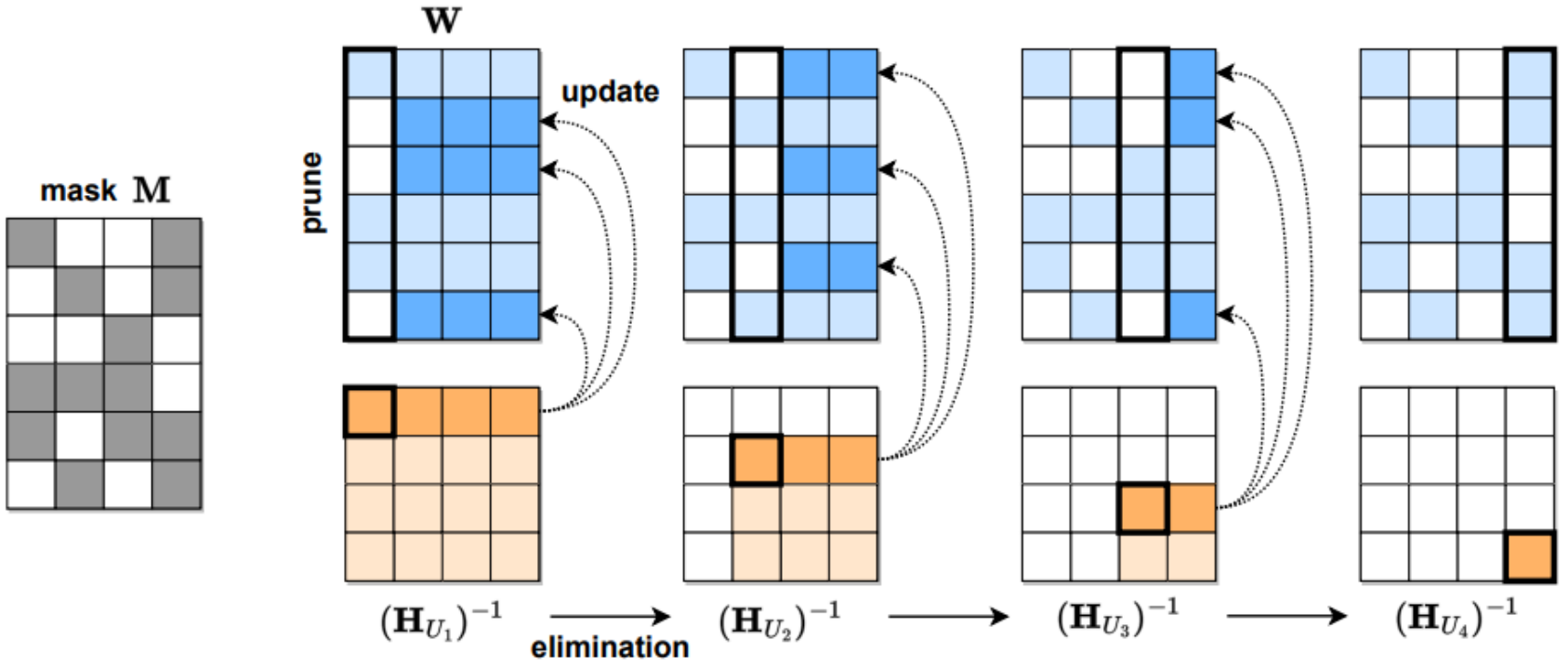


- Row-Hessian Challenge
  - Solving each row i requires the individual inversion of $\mathbf{H}_{M_i}$ matrix
    - For a given pruning mask M_i, we need to recompute $(\mathbf{H}_{M_i})^{-1}$
    - Cannot reuse $\mathbf{H}^{-1}$ because $(\mathbf{H}_{M_i})^{-1} \neq (\mathbf{H}^{-1})_{M_i}$
- Mask selection and weight reconstruction are separated
- Iterative & Partial Weight Update

SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot [Frantar et al., ICML 2023]

SparseGPT

- Hessian Synchronization
  - Prune column by column, update only right-side, freeze left
    - Always the same inverse Hessian in all rows.



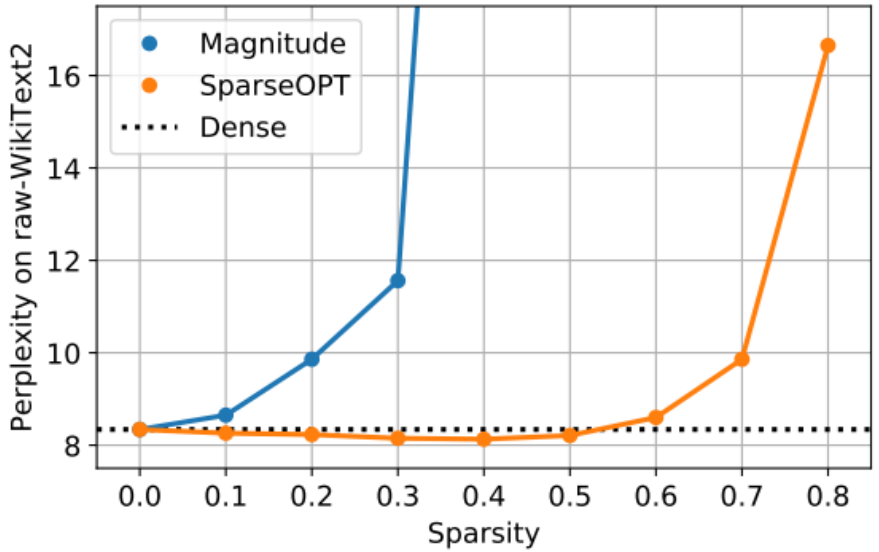
SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot [Frantar et al., ICML 2023]

SparseGPT

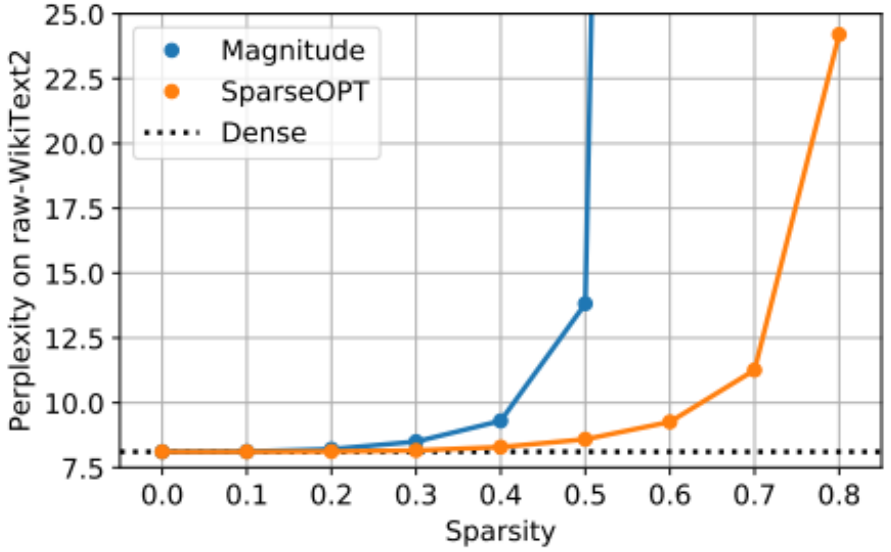
- Achieve significantly higher accuracy compared to the Magnitude Pruning.
- OPT-175B and BLOOM-176B can reach 60% unstructured sparsity with negligible increase in perplexity in under 4.5 hours.

https://www.lakera.ai/blog/large-language-model-evaluation

OPT-175B Uniform Unstructured Sparsity



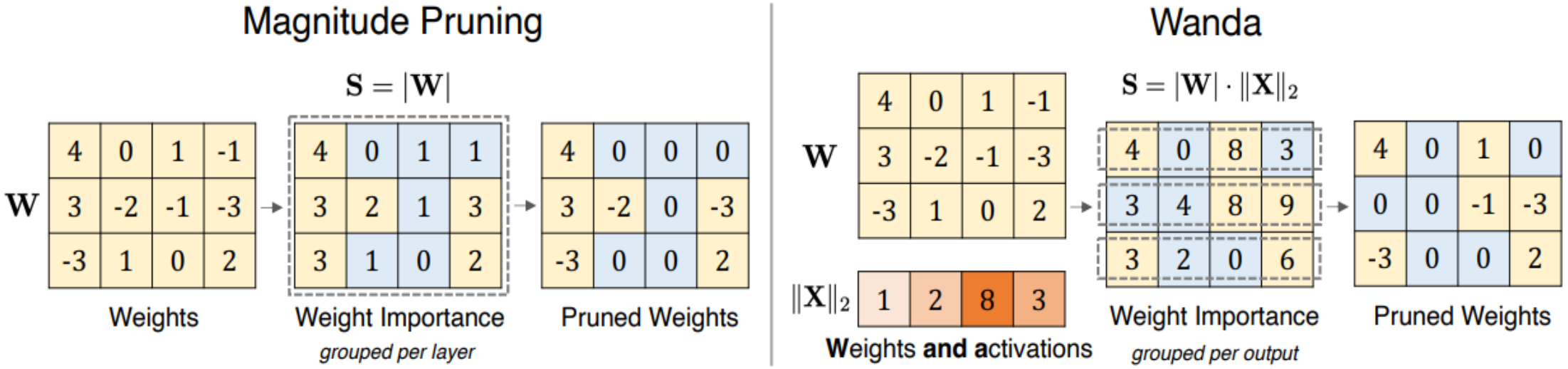
BLOOM-176B Uniform Unstructured Sparsity



SparseGPT: Massive Language Models Can Be Accurately Pruned in One-Shot [Frantar et al., ICML 2023]

Wanda

- Simple magnitude-based unstructured pruning
- Multiply the weight magnitudes by the norm of their associated input activations



A Simple and Effective Pruning Approach for Large Language Models [Sun et al., ICLR 2024]

Wanda

- Exceed the results of SparseGPT while reducing complexity

Method	Weight Update	Calibration Data	Pruning Metric $\mathbf{S}_{ij}$	Complexity
Magnitude	✗	✗	$\lvert\mathbf{W}_{ij}\rvert$	$O(1)$
SparseGPT	✓	✓	$\left[\lvert\mathbf{W}\rvert^2/\mathrm{diag}\left[(\mathbf{X}\mathbf{X}^T+\lambda\mathbf{I})^{-1}\right]\right]_{ij}$	$O(d^3_{\texttt{hidden}})$
Wanda	✗	✓	$\lvert\mathbf{W}_{ij}\rvert \cdot \lVert\mathbf{X}_j\rVert_2$	$O(d^2_{\texttt{hidden}})$

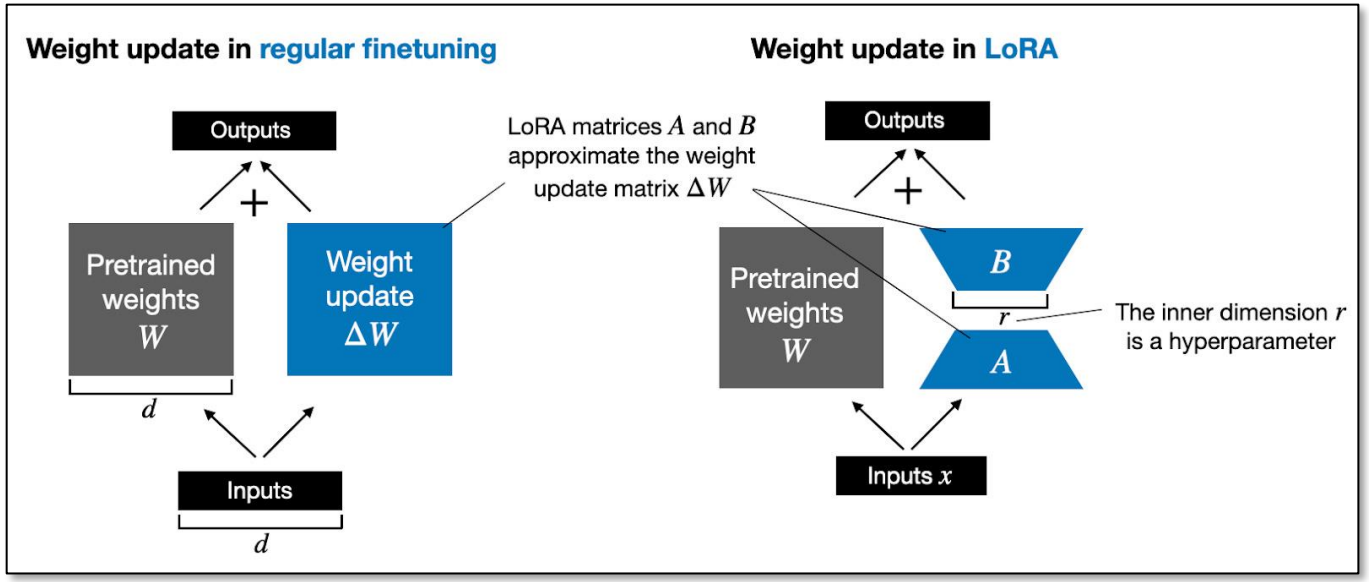
Zero-shot Accuracy			LLaMA				LLaMA-2		
Method	Weight Update	Sparsity	7B	13B	30B	65B	7B	13B	70B
Dense	-	0%	59.99	62.59	65.38	66.97	59.71	63.03	67.08
Magnitude	✗	50%	46.94	47.61	53.83	62.74	51.14	52.85	60.93
SparseGPT	✓	50%	**54.94**	58.61	63.09	66.30	**56.24**	60.72	**67.28**
Wanda	✗	50%	54.21	**59.33**	**63.60**	**66.67**	**56.24**	**60.83**	67.03

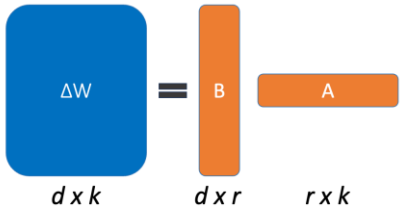
A Simple and Effective Pruning Approach for Large Language Models [Sun et al., ICLR 2024]

Parameter-Efficient Fine-Tuning (PEFT)

LoRA

- Instead of updating the full model weights, update a small low-rank component
  - $W_0x + \Delta Wx = W_0x + BAx$
- If $x \in \mathbb{R}^d$,
  - ΔW: $d \times d$ parameters
  - A and B: $d \times r$ parameters
  - r: decomposition rank
  - $d \times d \gg 2 \times d \times r, \text{if } d \gg r$
- Benefits:
  - Speed up fine-tuning by skipping gradient computation
  - Save fine-tuning memory by reducing optimizer states
  - Prevent catastrophic forgetting
  - Low-rank weights can be fused

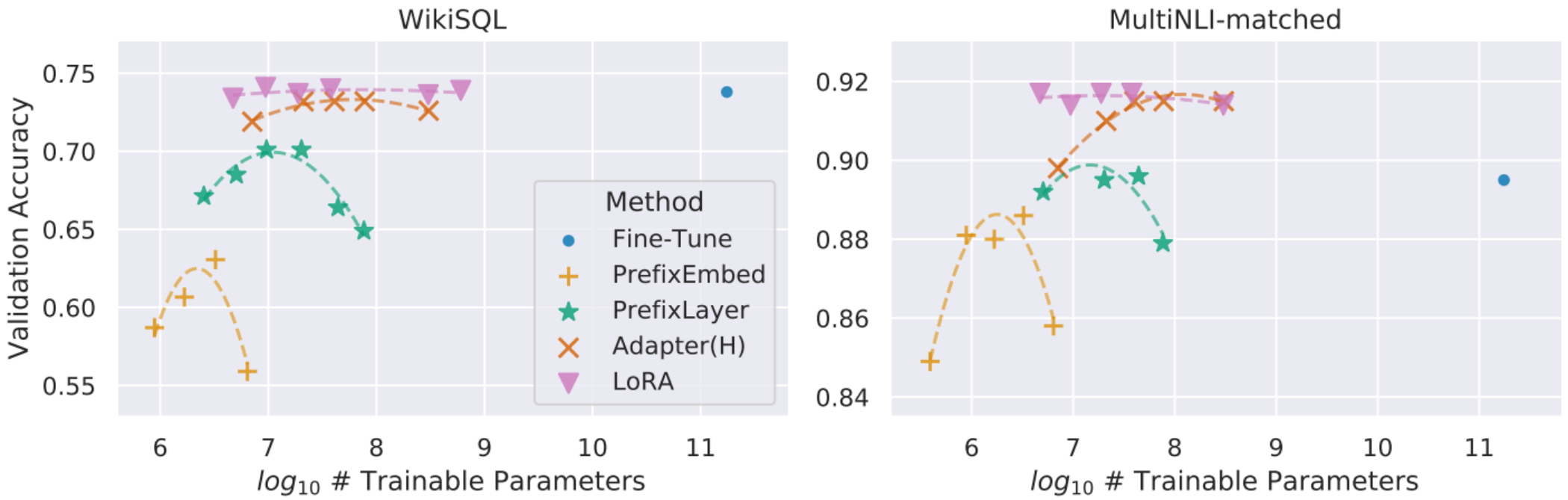




LoRA: Low-Rank Adaptation of Large Language Models [Hu et al, ICLR 2024]

LoRA

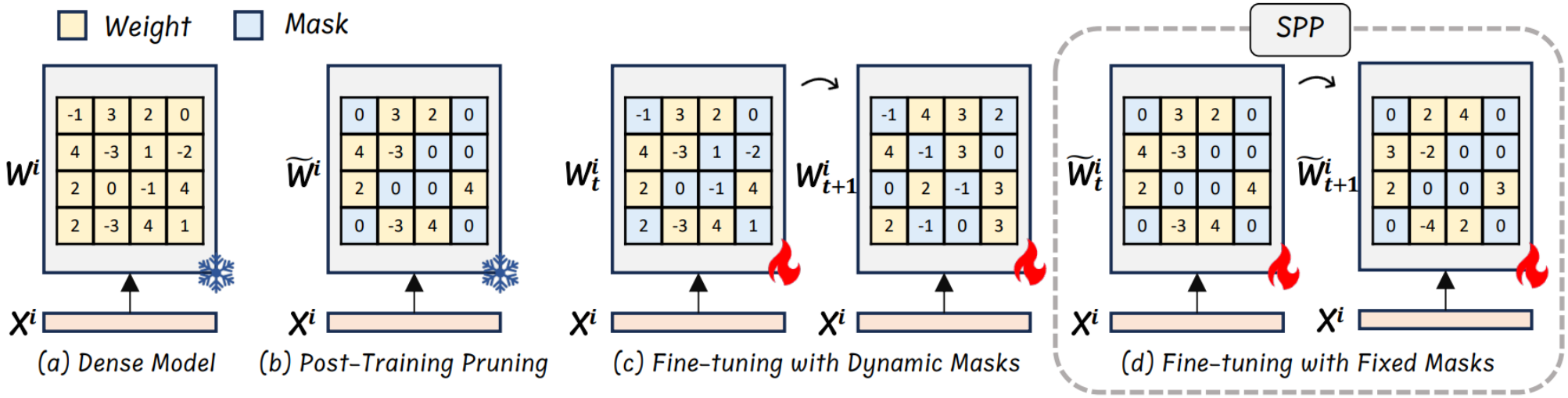
- GPT-3 175B, validation accuracy vs. number of trainable parameters
- Better scalability and task performance than several adaptation methods.



LoRA: Low-Rank Adaptation of Large Language Models [Hu et al, ICLR 2024]

SPP: Sparsity-Preserved PEFT

- PTP approaches (SparseGPT, Wanda) focuses on minimizing the errors between pruned and original dense models, without retraining.
  - Fail to maintain the performance of pre-trained models at even moderate sparsity levels (e.g., 50%)
  - Too long time for fine-tuning, mask changes
- We need an efficient retraining technique for sparse LLM models.
  - LoRA results in dense matrices and destroys model sparsity.

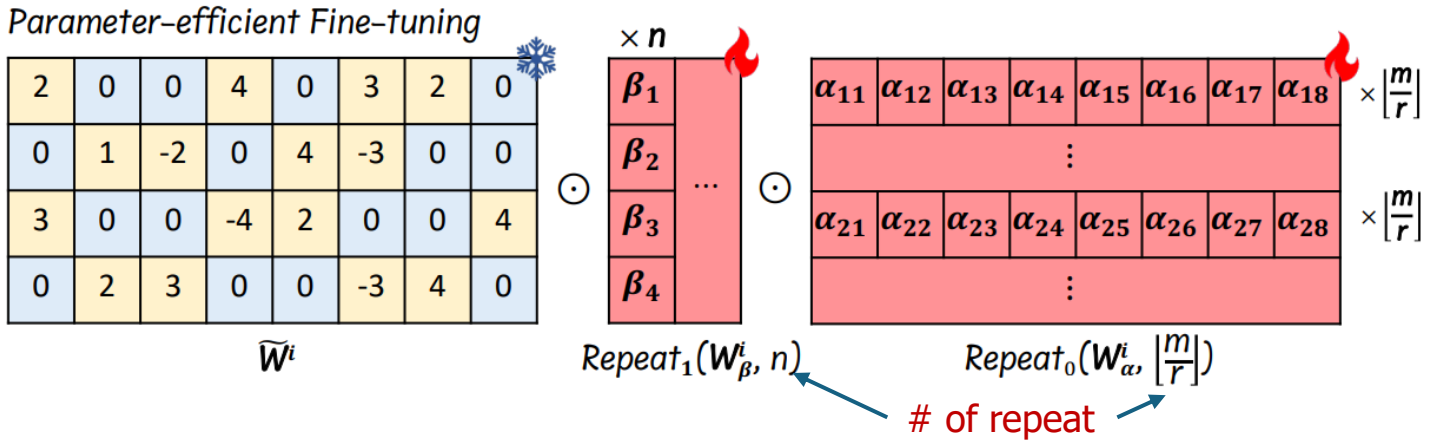


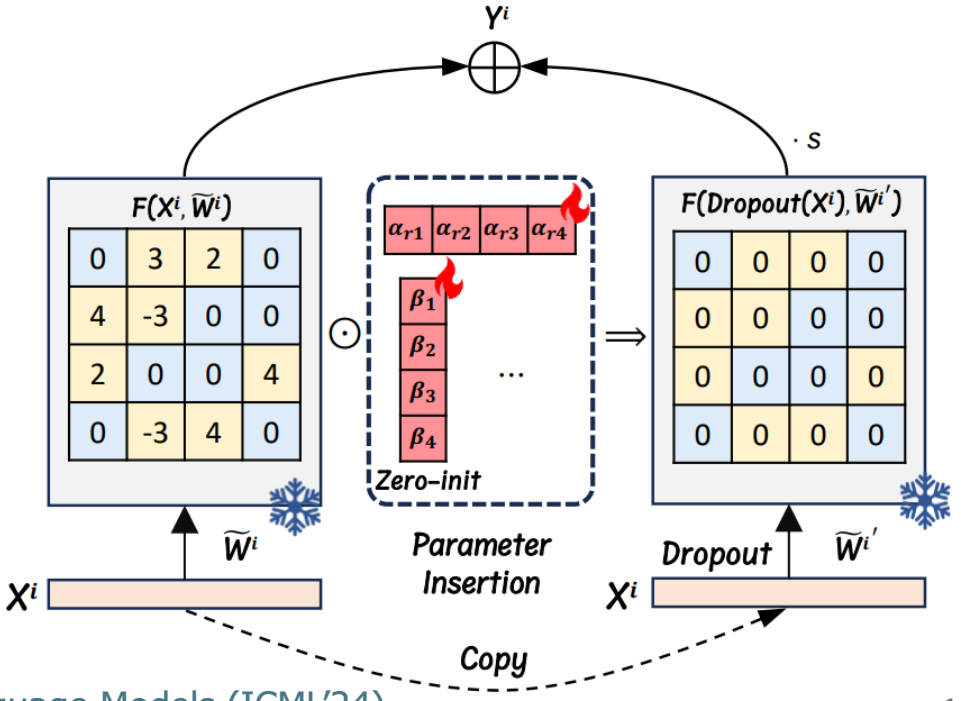
SPP: Sparsity-Preserved Parameter-Efficient Fine-Tuning for Large Language Models (ICML'24)

SPP: Sparsity-Preserved PEFT

- LoRA adds new parameters to sparse matrices
- SPP multiplies new learnable parameters to $\widetilde{W}^i \in R^{m \times n}$
  - $W_\alpha^i \in R^{r \times n}$ and $W_\beta^i \in R^{m \times 1}$, $(m + rn)$ additional parameters
  - W_α^i to the weights of each column, W_β^i to each row.
  - r determines capacity.
  - does not destroy the sparse structure

$$\mathbf{Y}^i = \mathbf{F}(\mathbf{X}^i, \widetilde{\mathbf{W}}^i) + s \cdot \mathbf{F}(\mathrm{Dropout}(\mathbf{X}^i), \widetilde{\mathbf{W}}^{i'})$$





SPP: Sparsity-Preserved Parameter-Efficient Fine-Tuning for Large Language Models (ICML'24)

SPP: Sparsity-Preserved PEFT

LLaMA	Method	Sparsity	BoolQ	RTE	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA	Average
7B	None	Dense	75.11	66.43	56.96	69.85	75.25	41.89	34.40	59.98
	SparseGPT	Unstructured 50%	73.36	57.76	51.44	68.03	70.45	36.35	28.40	55.11
	SparseGPT+**SPP**	Unstructured 50%	72.84	65.70	56.40	67.88	72.35	41.04	32.80	**58.43**
	SparseGPT	2:4	70.09	57.76	43.37	63.46	61.62	29.27	22.60	49.74
	SparseGPT+**SPP**	2:4	72.39	59.57	53.33	64.17	68.39	37.54	26.80	**54.60**
	Wanda	Unstructured 50%	71.01	55.23	51.90	66.22	69.36	36.95	28.60	54.18
	Wanda+**SPP**	Unstructured 50%	70.86	66.06	55.92	67.64	72.81	41.64	32.00	**58.13**
	Wanda	2:4	69.27	51.26	42.07	62.67	60.52	27.99	24.60	48.34
	Wanda+**SPP**	2:4	71.19	63.90	52.77	64.88	68.18	37.03	30.00	**55.42**
13B	None	Dense	77.98	70.40	59.92	72.61	77.36	46.50	33.20	62.57
	SparseGPT	Unstructured 50%	76.54	62.09	54.94	71.59	72.35	41.64	32.20	58.76
	SparseGPT+**SPP**	Unstructured 50%	79.20	64.62	59.27	70.32	74.83	46.59	34.60	**61.35**
	SparseGPT	2:4	70.80	56.68	48.09	69.22	66.88	36.26	26.20	53.45
	SparseGPT+**SPP**	2:4	77.65	63.54	56.55	69.69	71.21	40.96	32.60	**58.89**
	Wanda	Unstructured 50%	76.27	62.82	55.78	71.98	73.32	43.77	31.80	59.39
	Wanda+**SPP**	Unstructured 50%	78.29	66.43	58.88	70.32	75.59	46.93	34.40	**61.55**
	Wanda	2:4	70.21	53.79	46.78	68.82	65.74	33.70	26.20	52.18
	Wanda+**SPP**	2:4	75.99	58.12	56.07	68.90	70.37	40.53	32.40	**57.48**

SPP: Sparsity-Preserved Parameter-Efficient Fine-Tuning for Large Language Models (ICML'24)

LLM Quantization

- CNN Quantization
  - FP32 → Int8 → Int4 → 2/1b (XNOR-Net, BinaryNet)
  - QAT is reasonable
  - Bert quantization also followed CNN's approaches
- LLM Quantization
  - FP16 → Int8 → Int4 → 1b (BitNet)
  - PTQ is popular (Llama-2 have the cost of >$20 million (278억) to train)
    - QLora: Quantization at fine-tuning for downstream task
  - Weight-only, Weight-Activation, KV Cache

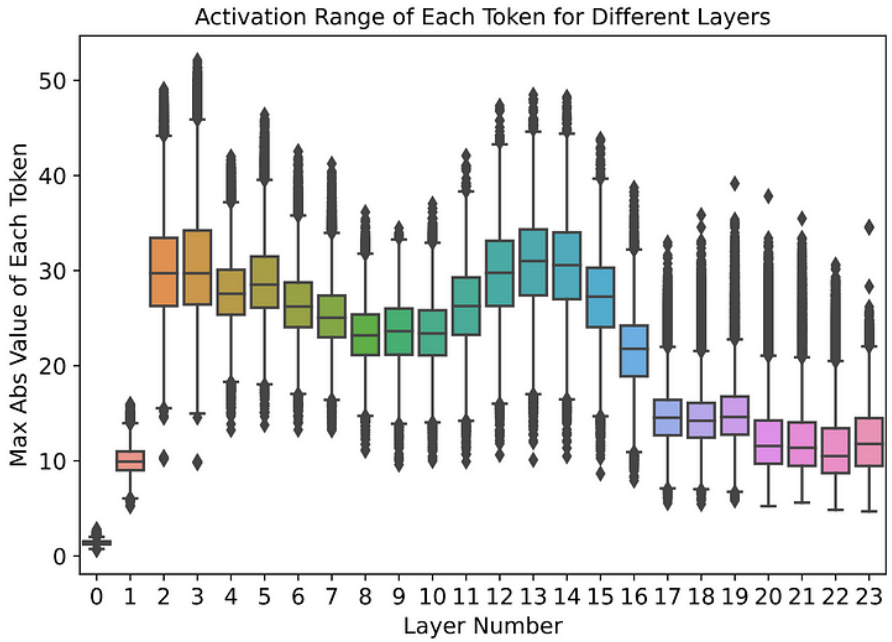
Post-Training Quantization (PTQ)

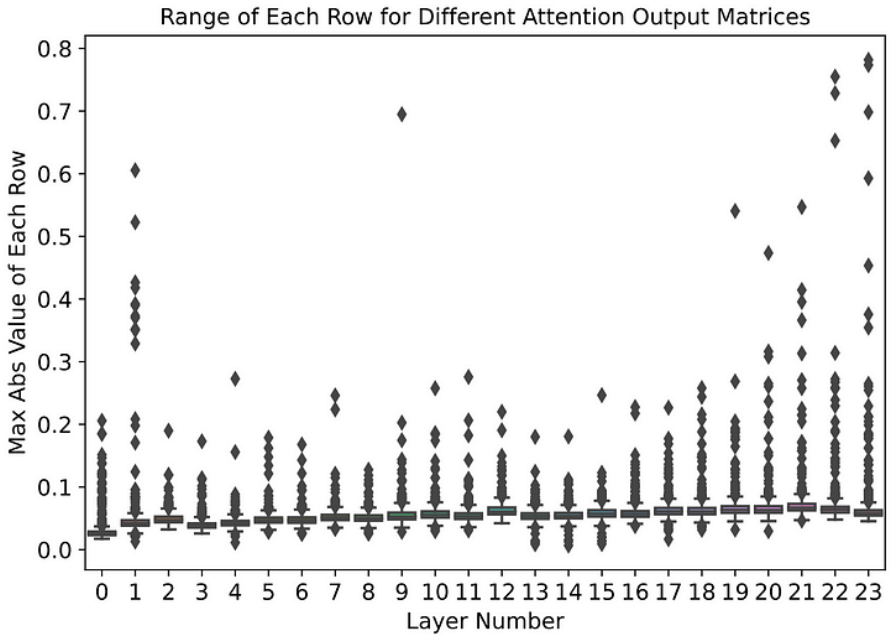
- Weight-Activation Co-Quantization
  - ZeroQuant (MS)
  - LLM.int8() (UW)
  - SmoothQuant (Nvidia, MIT)
  - OmniQuant (OpenGVLab)
  - QuaRot (ETH Zurich)
- Weight-Only Quantization
  - GPTQ (Neural Magic)
  - SpQR (UW)
  - AWQ (Nvidia, MIT)

How to handle Outliers?

ZeroQuant

- Highly dynamic range in the activations (tokens) between layers
  - A very diverse distribution of activations depending on input token
  - A static quantization scheme (applied to all tokens/samples) would lead to significant accuracy drop.
- Weight matrices have long-tailed ranges
  - Each head is independently trained and has significantly different distributions.





ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers [Z Yao et al., NeurIPS 2024]

ZeroQuant

- INT8 quantization for both weights and activations (W8A8)
  - **Group-wise** Quantization for **Weights**
    - Weight matrix is divided into g groups, and each group is quantized differently
    - By appropriately setting g, it is possible to quantize differently
  - **Token-wise** Quantization for **Activations**
    - Calibrate token-wise min/max ranges dynamically during inference
- Layer-by-layer knowledge distillation (LKD) approach
  - The original (unquantized) version is the teacher model
- Fused operation backend that hides quantization/dequantization overhead
- Open-sourced as part of the DeepSpeed library, which is integrated into Hugging Face's Transformers library.



Figure 3: The illustration of normal (left) and our fused (right) INT8 GeMM.

ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers [Z Yao et al., NeurIPS 2024]

ZeroQuant

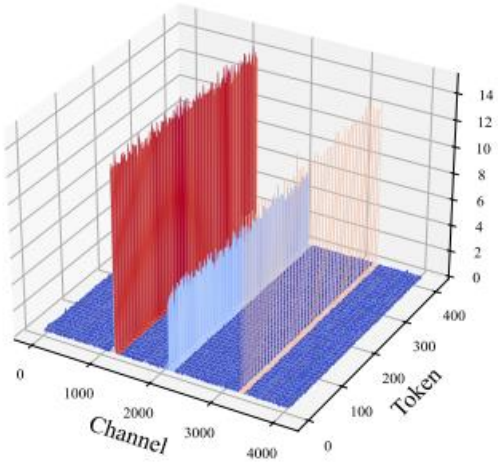
- Similar accuracy to the FP16 model with 5.2x better efficiency.
- LKD compresses the less sensitive weights to 4 bits without requiring training data, resulting in a 3x smaller memory footprint than the FP16 model.

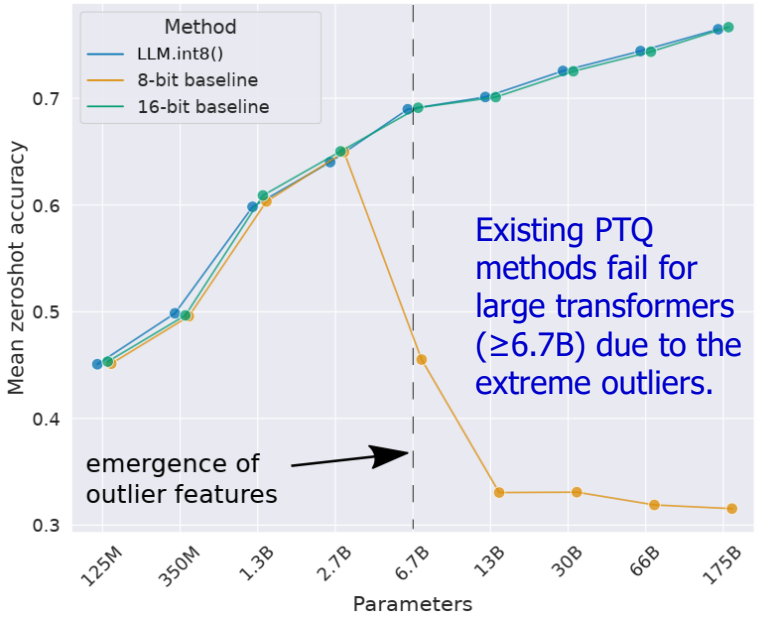
Precision (Method)	Lambada (↑)	PIQA (↑)	OpenBookQA (↑)	RTE (↑)	ReCoRd (↑)	Ave. 19 Tasks (↑)	Wikitext-2 (↓)	Time Cost
W16A16	49.3	66.3	29.4	53.8	75.1	38.9	21.5	N/A
W8A8 (PTQ)	42.6	64.1	28.0	53.1	67.5	37.8	26.2	7 mins
W8A8 (ZeroQuant)	51.0	66.5	29.2	53.4	74.9	38.7	21.7	0
W4/8A16 (PTQ)	0.00	51.4	30.2	52.7	16.1	28.9	1.76e5	7 mins
W4/8A16 (ZeroQuant)	10.1	58.5	27.2	52.0	56.5	33.5	88.6	0
W4/8A16 (ZeroQuant-LKD)	39.8	63.8	29.4	53.1	70.1	37.0	30.6	1.1 hours
W4/8A8 (ZeroQuant)	10.5	57.7	28.0	52.7	55.3	33.4	92.1	0
W4/8A8 (ZeroQuant-LKD)	37.4	61.8	28.2	53.1	68.5	36.6	31.1	1.1 hours

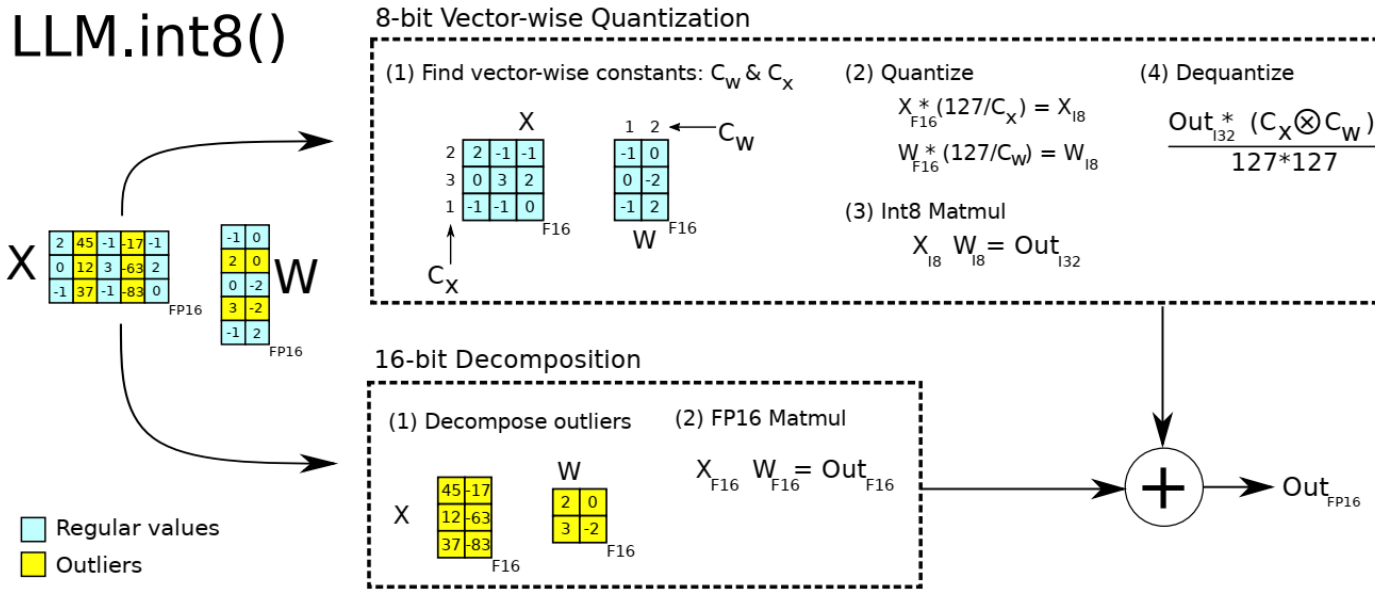
ZeroQuant: Efficient and Affordable Post-Training Quantization for Large-Scale Transformers [Z Yao et al., NeurIPS 2024]

LLM.int8() aka GPT3.int8()

- 8-bit integer PTQ method (W8A8)
- Vector-wise quantization granularity.
  - scaling by row and column-wise absolute maximum (C_x, C_w)
- Separate outlier and normal values into two matrices
  - Outlier feature matrices are multiplied in 16-bit
- Open-sourced in the bitsandbytes library (Hugging Face)



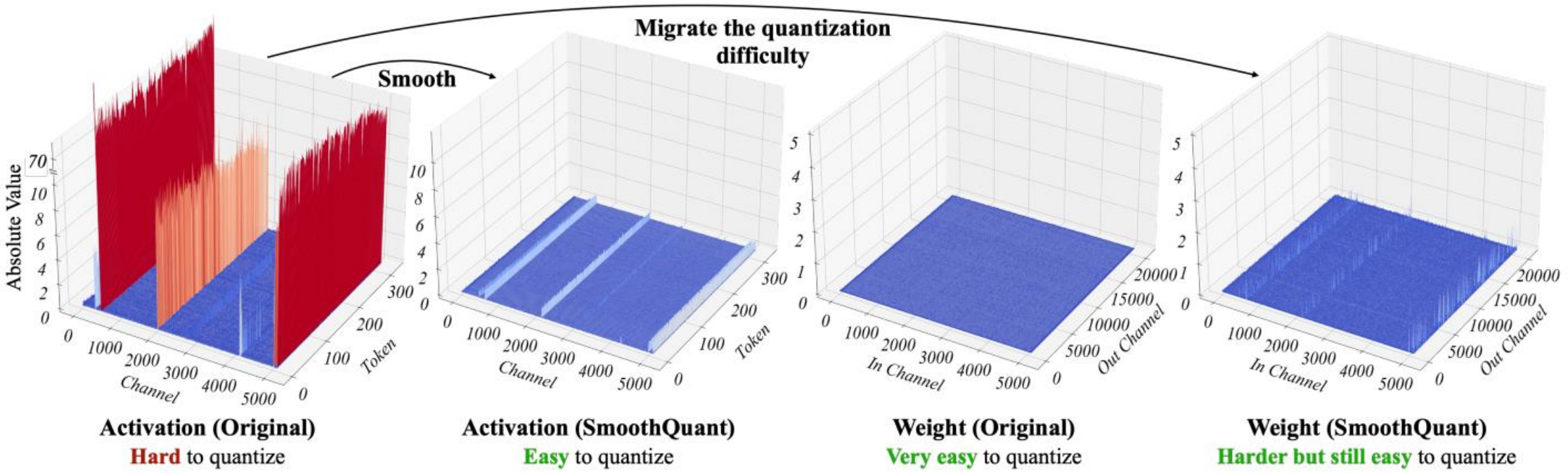




LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale [NeurIPS'22]

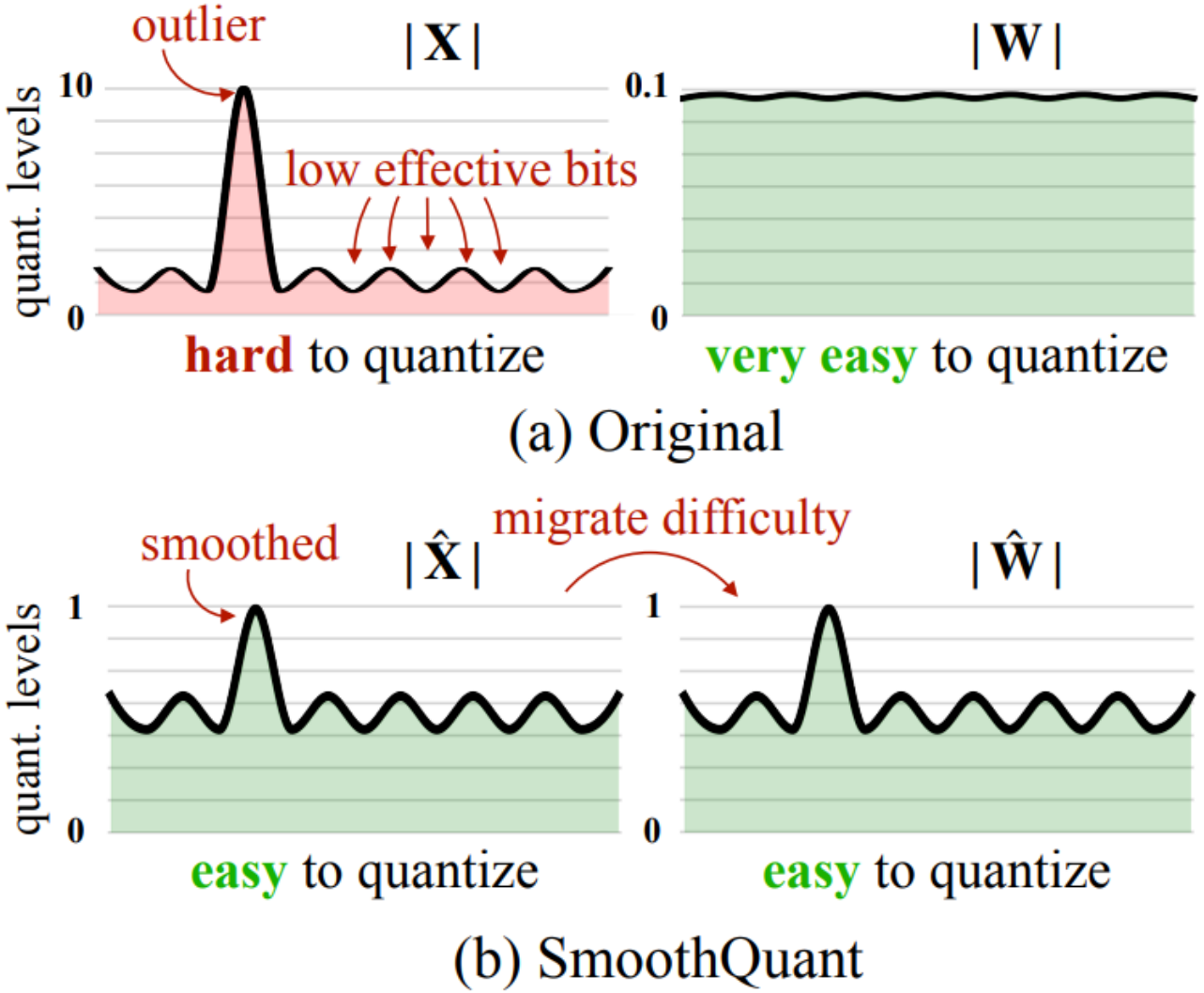
SmoothQuant

- Outliers persist in fixed channels.
- Transfer appropriate outliers from activation to weight offline through channel-wise smoothing scales



SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

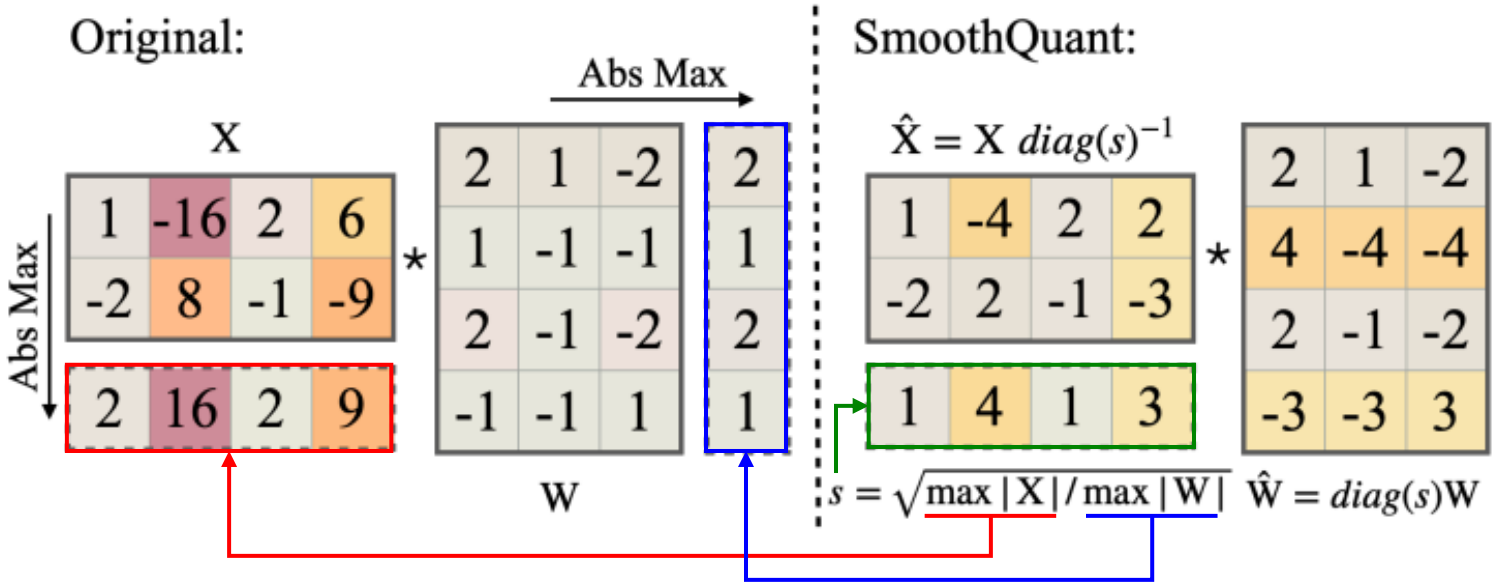
SmoothQuant



SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

SmoothQuant

- Smoothing activation to reduce quantization error

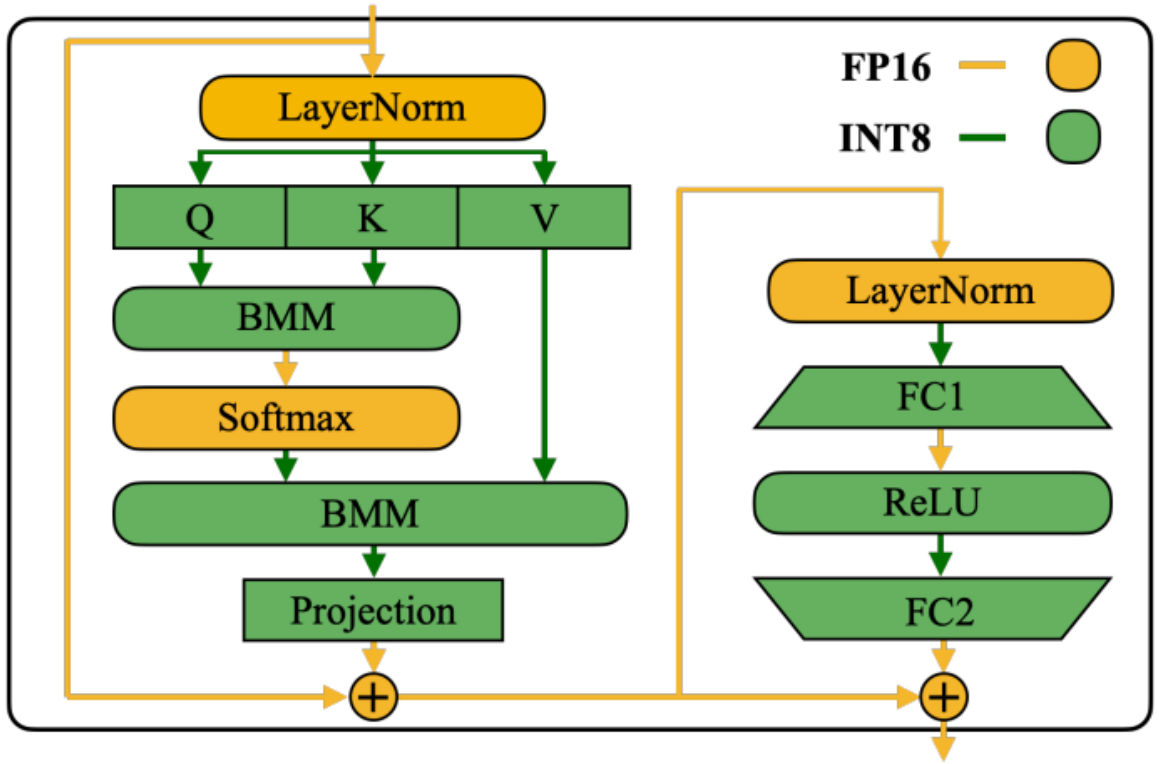


$$\mathbf{Y} = \mathbf{XW} = (\mathbf{X}diag(s)^{-1}) \cdot (diag(s)\mathbf{W}) = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

SmoothQuant

- Efficient System Implementation
  - All compute-intensive operators (Linears, Batched Matmul) use INT8



LayerNorm and SoftMax layers are still computed in FP16

SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

SmoothQuant

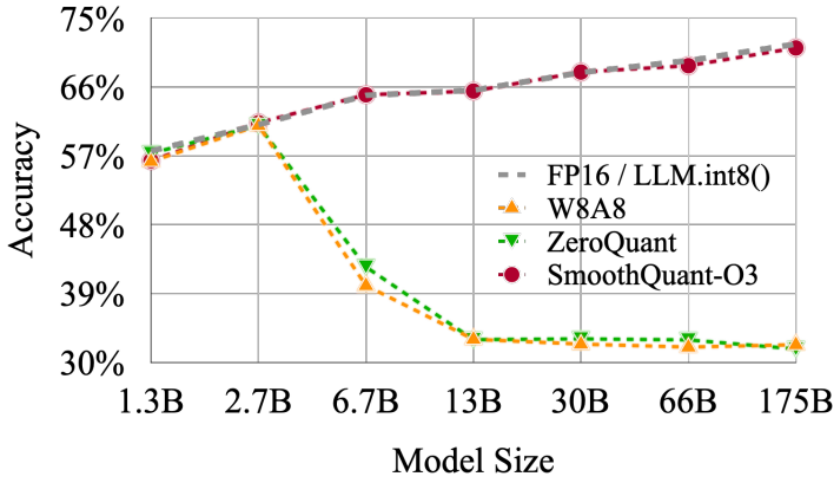
- Well maintains the accuracy without fine-tuning.
- Can both accelerate inference and halve the memory footprint.

Method	Weight	Activation
W8A8	per-tensor	per-tensor dynamic
ZeroQuant	group-wise	per-token dynamic
`LLM.int8()`	per-channel	per-token dynamic+FP16
Outlier Suppression	per-tensor	per-tensor static
SmoothQuant-O1	per-tensor	per-token dynamic
SmoothQuant-O2	per-tensor	per-tensor dynamic
SmoothQuant-O3	per-tensor	per-tensor static

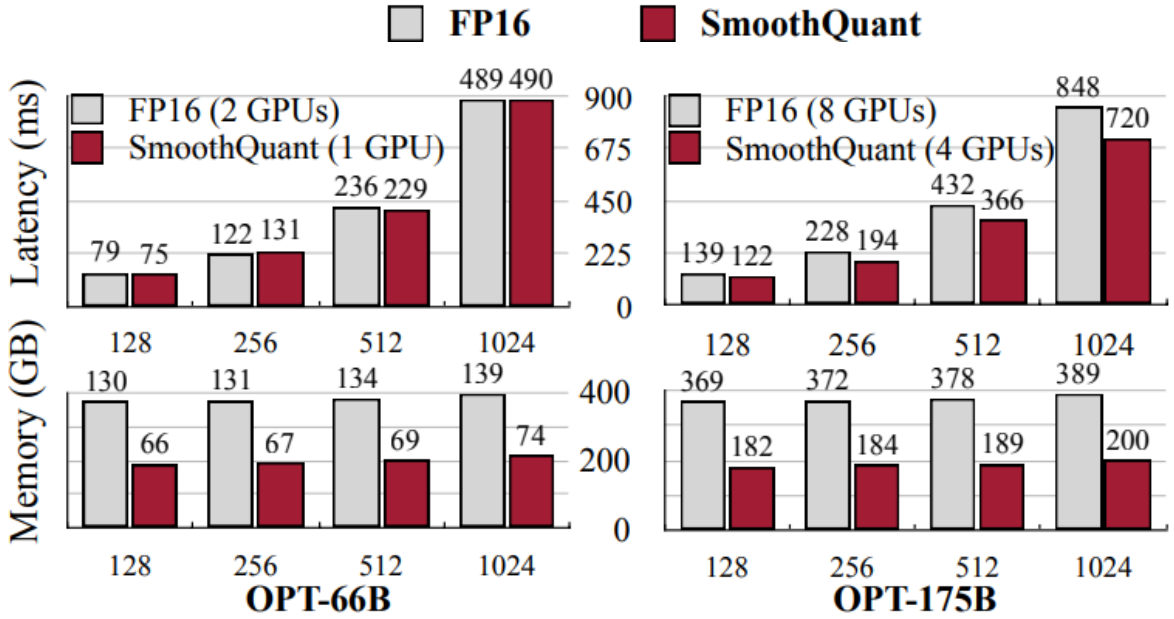
→ The scaling factor is calibrated offline

Wiki PPL↓	7B	13B	30B	65B
FP16	11.51	10.05	7.53	6.17
W8A8 SmoothQuant	11.56	10.08	7.56	6.20

losslessly quantize LLaMA families



Equal performance to LLM.int8() without the need for mixed-precision decomposition of the activation.



SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., ICML 2023]

OmniQuant

- Learnable Weight Clipping (LWC)
  - $\gamma, \beta \in [0,1]$: learnable clipping strengths for the upper and the lower bound of weights

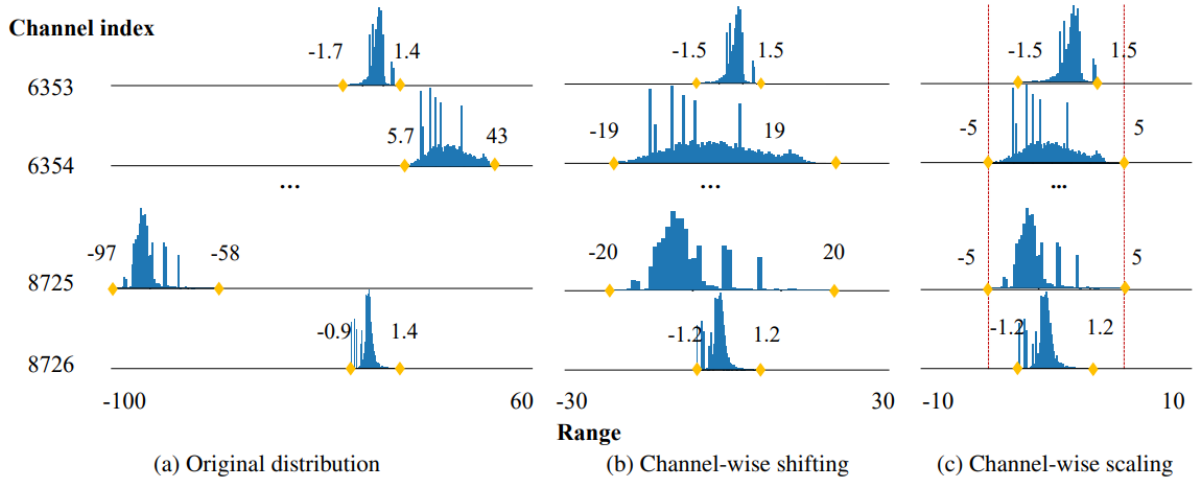
$$\mathbf{W_q} = \text{clamp}(\lfloor \frac{\mathbf{W}}{h} \rceil + z, 0, 2^N - 1), \text{where } h = \frac{\gamma \max(\mathbf{W}) - \beta \min(\mathbf{W})}{2^N - 1}, z = -\lfloor \frac{\beta \min(\mathbf{W})}{h} \rceil \tag{2}$$

- Learnable Equivalent Transformation (LET)
  - Apply channel-wise scaling and channel-wise shifting to manipulate the activation distribution

$$\mathbf{Y} = \mathbf{X}\mathbf{W} + \mathbf{B} = \underbrace{[(\mathbf{X} - \delta) \oslash s]}_{\tilde{\mathbf{X}}} \cdot \underbrace{[s \odot \mathbf{W}]}_{\tilde{\mathbf{W}}} + \underbrace{[\mathbf{B} + \delta\mathbf{W}]}_{\tilde{\mathbf{B}}}$$

$$\mathbf{Y} = Q_a(\tilde{\mathbf{X}})Q_w(\tilde{\mathbf{W}}) + \tilde{\mathbf{B}},$$

$$\mathbf{P} = \text{Softmax}(\mathbf{Q}\mathbf{K}^T) = \text{Softmax}((\underbrace{\mathbf{Q} \oslash s_a}_{\tilde{\mathbf{Q}}})(\underbrace{s_a \odot \mathbf{K}^T}_{\tilde{\mathbf{K}}^T}))$$

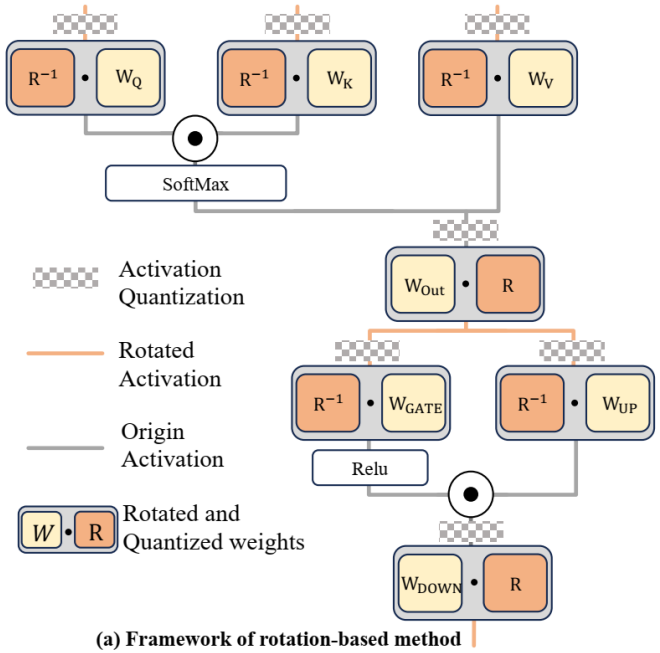


OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models [ICLR'24]

Outlier Suppression+ [EMNLP'23]

QuaRot, SpinQuant

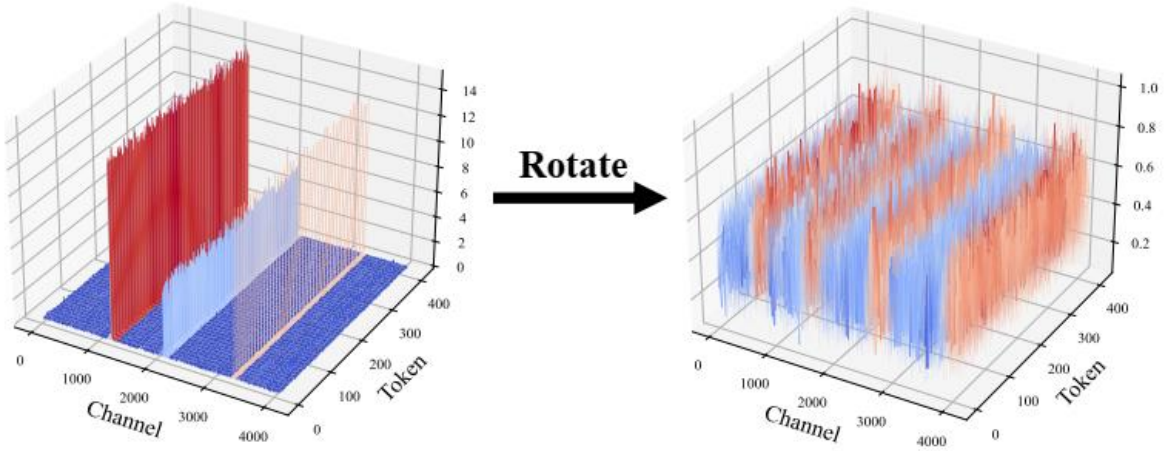
- Make the activations easy to quantize by rotating them using randomized Hadamard transformation (remove outlier features)
- Enable 4-bit LLM inference by quantizing all weights, activations, and KV caches (cf. SmoothQuant cannot implement A4W4)



(a) Framework of rotation-based method

$$\mathbf{Y} = (\mathbf{XR})\left(\mathbf{R^{-1}W^{-1}}\right) = \mathbf{XW^T}$$

A rotation matrix is an orthogonal matrix $\mathbf{R}$ satisfied $\mathbf{RR^T} = 1$ and $|\mathbf{R}| = 1$

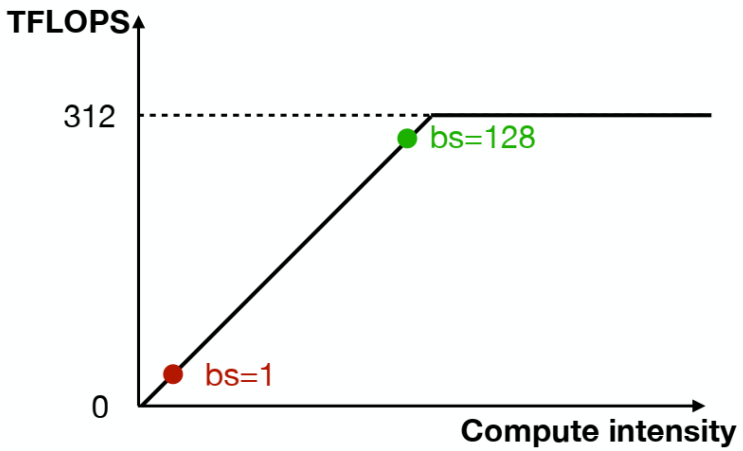


QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs [NeurIPS'24]

Rotated Runtime Smooth: Training-Free Activation Smoother for accurate INT4 inference [ICLR'25]

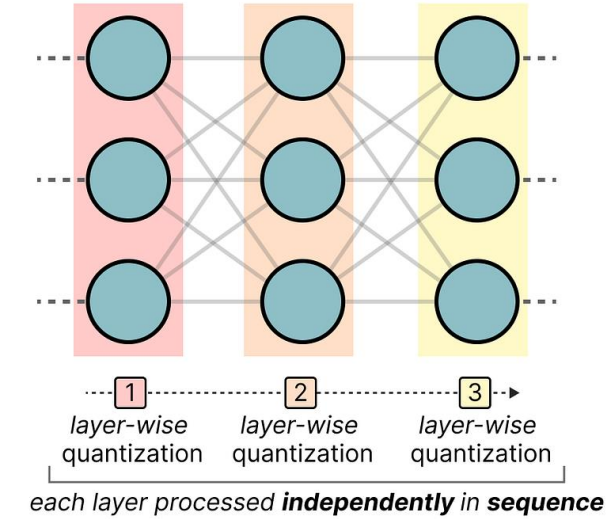
Weight-Only Quantization

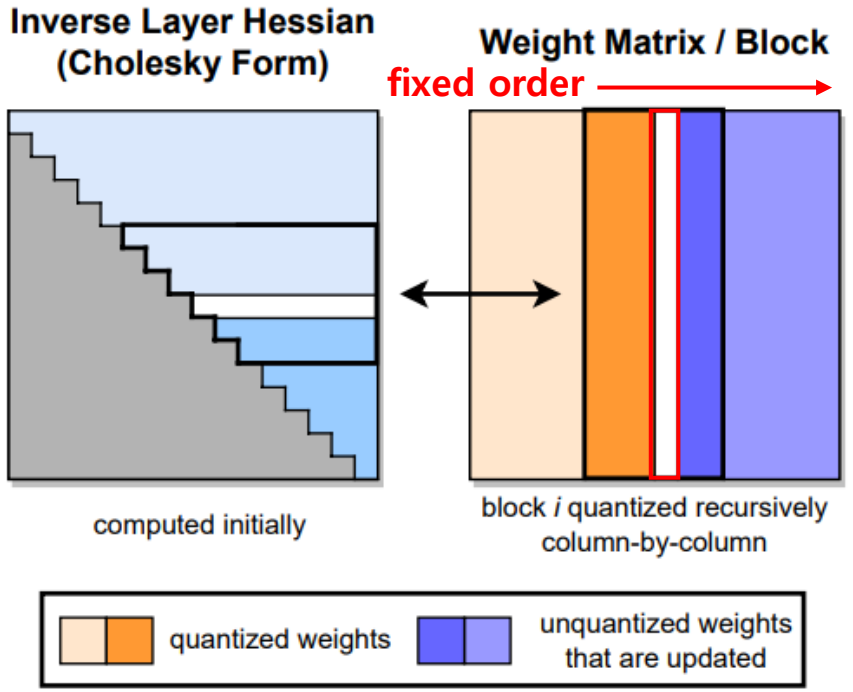
- W8A8 quantization is good for batch serving (e.g., batch size 128)
- But single-query LLM inference is highly memory-bounded
- We need low-bit weight-only quantization (e.g., W4A16)
  - No efficiency gains in the multiplication operations, due to the mixed-precision interactions with the non-quantized activations.
    - Reduced memory traffic can speed up inference
  - The reduced memory footprint is essential for reducing the size of the GPU hardware required for inference.
    - GPTQ ran OPT-175B for the first time on a single A100 GPU.



GPTQ

- Layer-Wise Quantization
- Based on OBQ (Optimal Brain Quantization)
  - Adjust the remaining unquantized weights at each step, compensating for the error introduced by quantizing the current weight.
- Quantize rows of W independently
- Quantize column-by-column
- Hessian $\mathbf{H}_F = 2\mathbf{X}_F\mathbf{X}_F^T$
- Perform the update of $\mathbf{H}_F^{-1}$ only d_{col} times (once per column)
- $\mathbf{H}_F^{-1}$ is calculated by Cholesky decomposition
- Lazy Batch-Updates
  - Apply the update to columns in the block





GPTQ

- 13B parameter models in a matter of minutes and 175B ones in a few hours.
  - cf., ZeroQuant reports a 3 hours runtime for a 1.3B model
- Accuracy (W3/W4 w/o retraining)
  - RTN (Round-To-Nearest) collapses, while GPTQ is still able to maintain good performance on most tasks.

OPT	13B	30B	66B	175B
Runtime	20.9m	44.9m	1.6h	4.2h
BLOOM	1.7B	3B	7.1B	176B
Runtime	2.9m	5.2m	10.0m	3.8h

Table 2: GPTQ runtime for full quantization of the 4 largest OPT and BLOOM models.

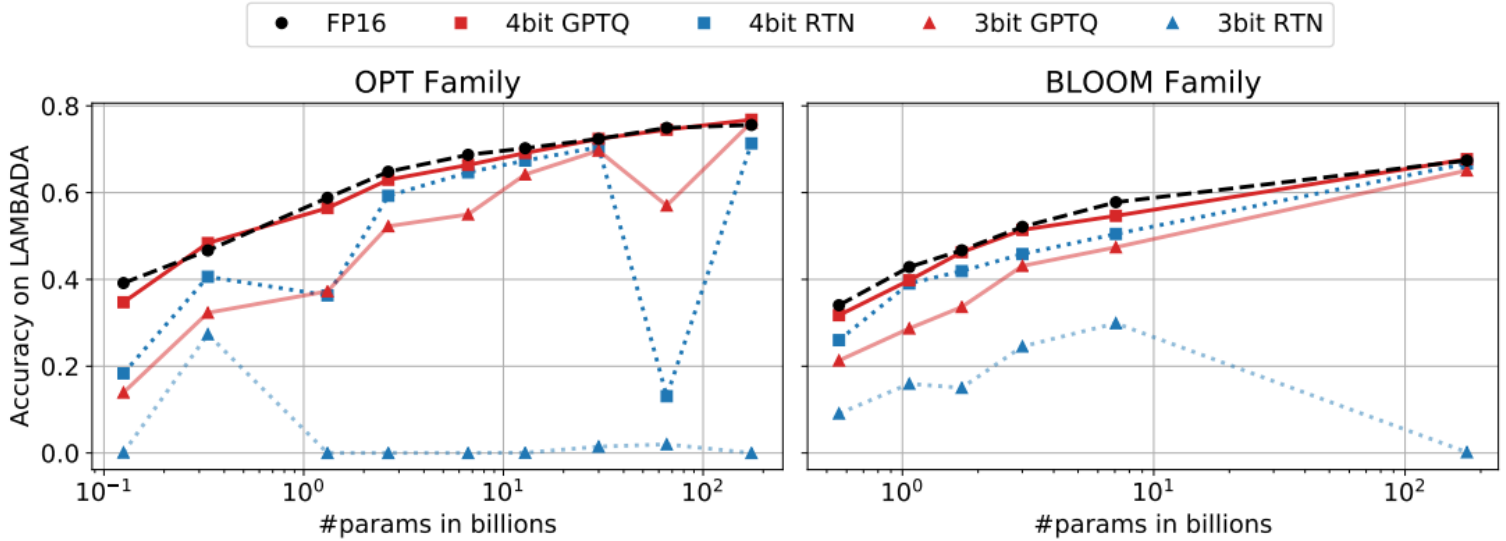


Figure 3: The accuracy of OPT and BLOOM models post-GPTQ, measured on LAMBADA.

GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers [Frantar et al., ICLR 2023]

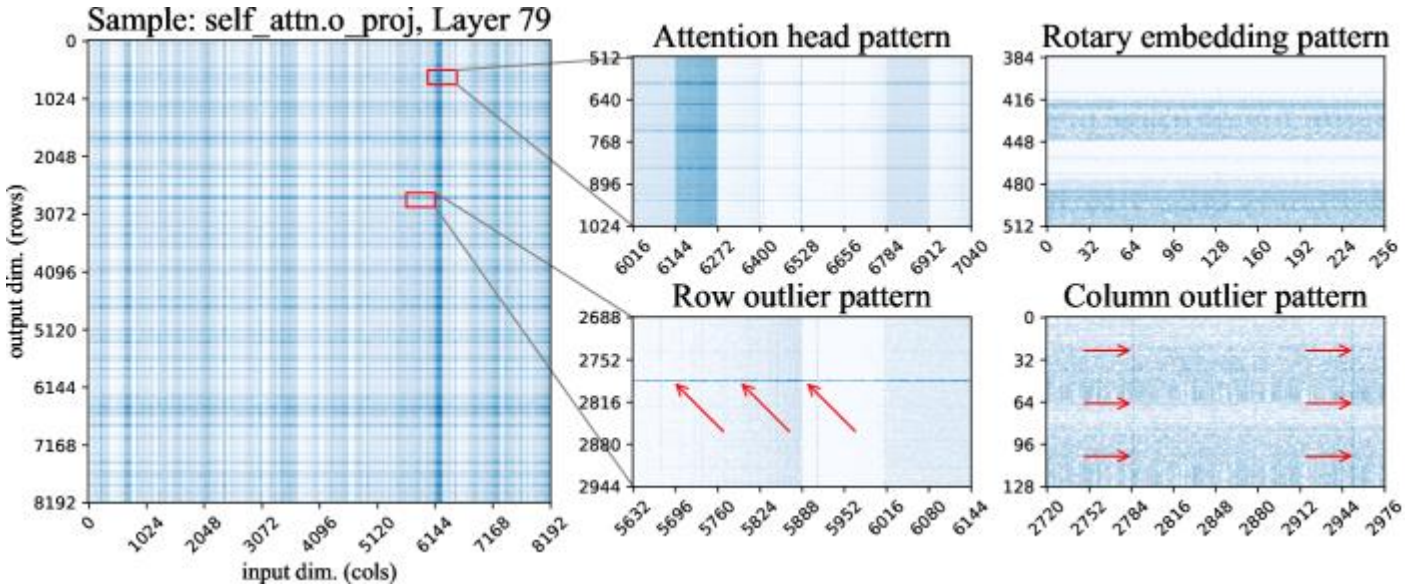
SpQR

- Isolate outlier weights and keep them in high precision.
- All other weights to 3-4 bits
- Parameter sensitivity

$$s_{ij} = \min_{W'} ||WX - W'X||_2^2 \quad \text{s.t.} \quad w'_{ij} = \text{quant}(w_{ij})$$

$$s_{ij} = \frac{(w_{ij} - \text{quant}(w_{ij}))^2}{2(XX^\top)^{-1}}.$$ GPTQ

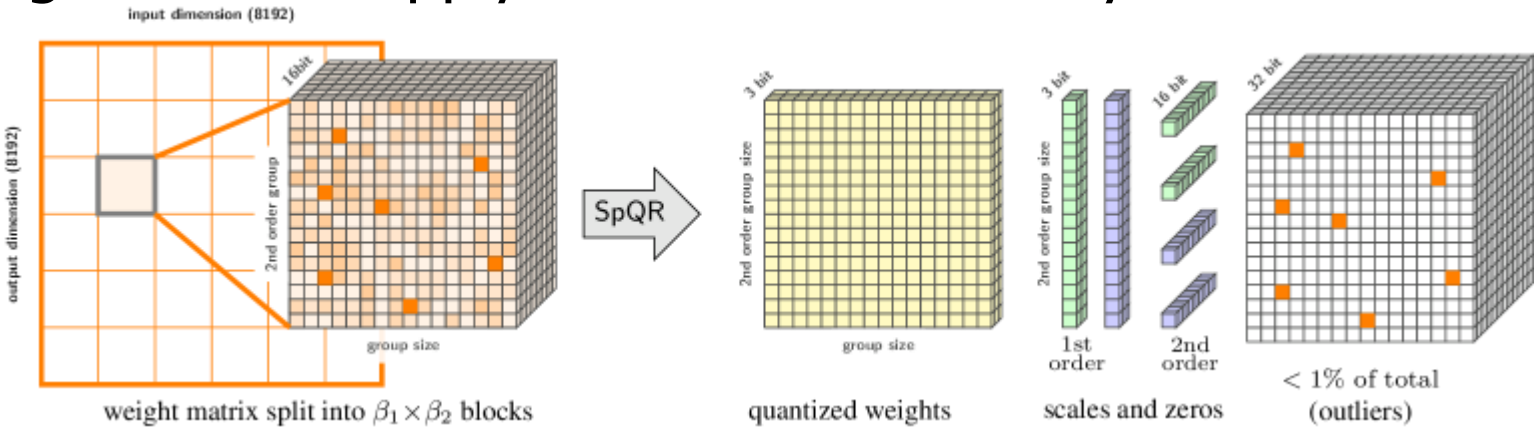
- Sensitive pattern
  - Row/column outliers
  - Sensitive attention heads
  - Rotary embedding pattern
  - Unstructured outliers



SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression [ICLR'24]

SpQR

- Bi-level quantization
  1. Group-level quantization (small group, 8~32 weights)
     - Each group has its own quantization scale and zero point
  2. Quantize the statistical data (scale and zero point)
- High-sensitivity outliers
  - Find and isolate outliers as 16-bit weights based on the sensitivity criterion
  - Storage method: apply CSR to save memory



SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression [ICLR'24]

SpQR

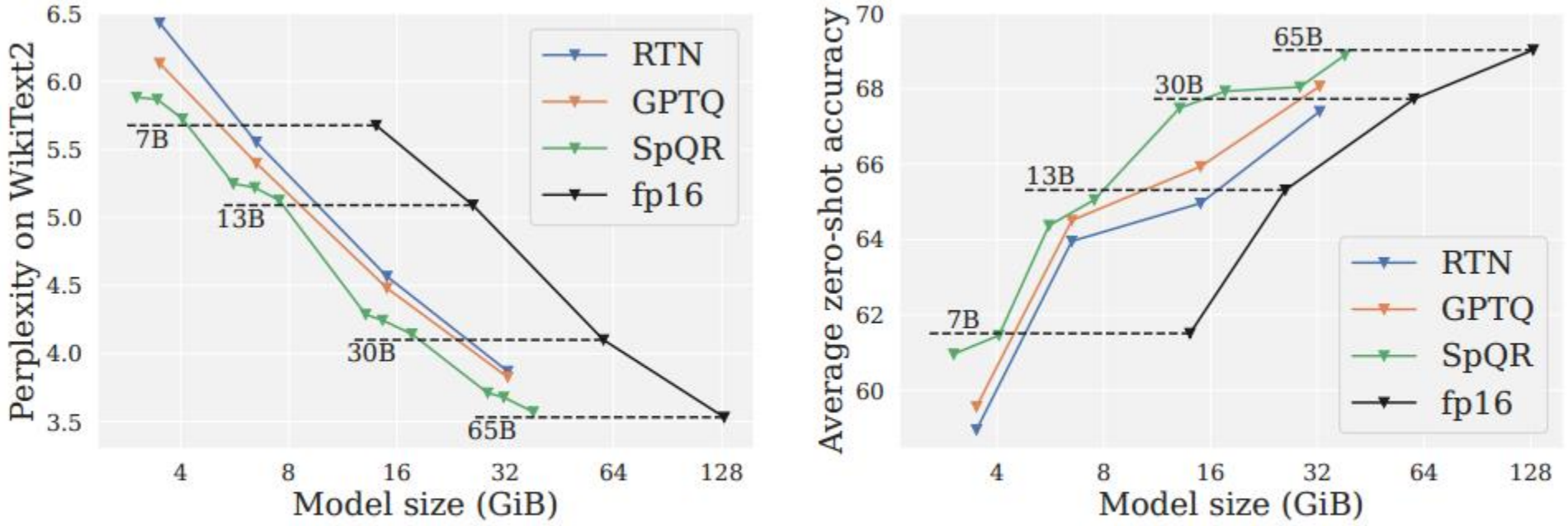
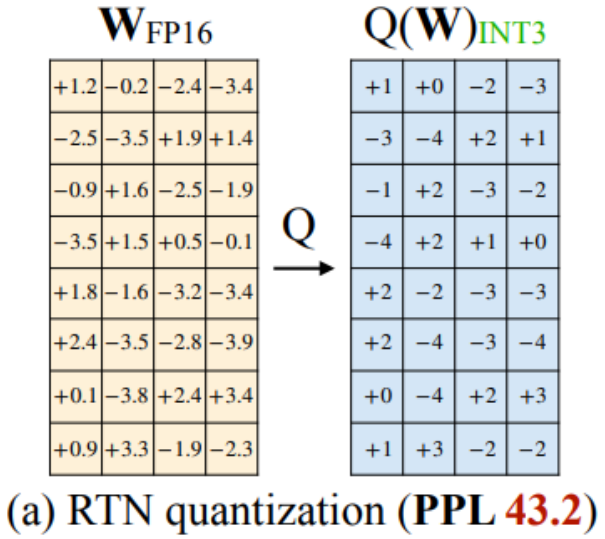


Figure 1: Compressed LLM performance for LLaMA models. (**left**) LM loss on WikiText2 vs model size. (**right**) Average performance on zero-shot tasks vs model size.

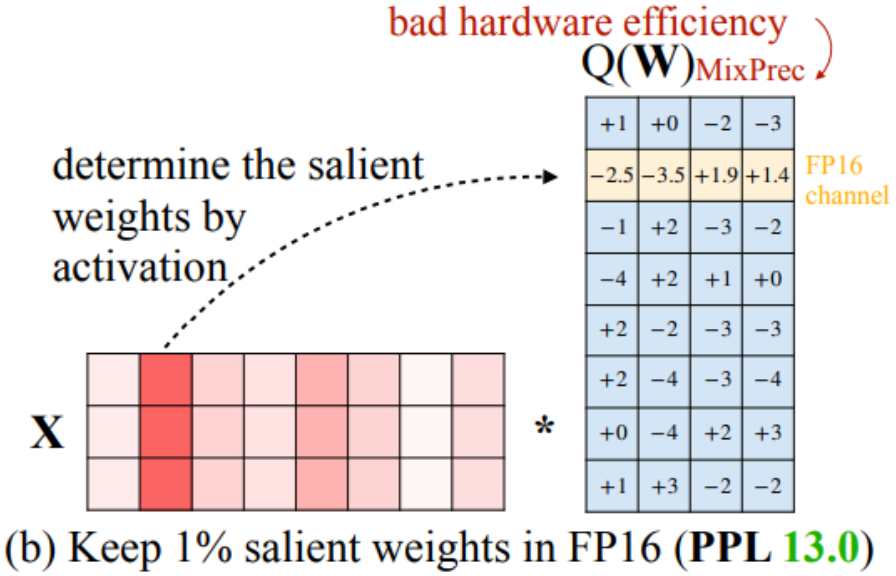
SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression [ICLR'24]

AWQ

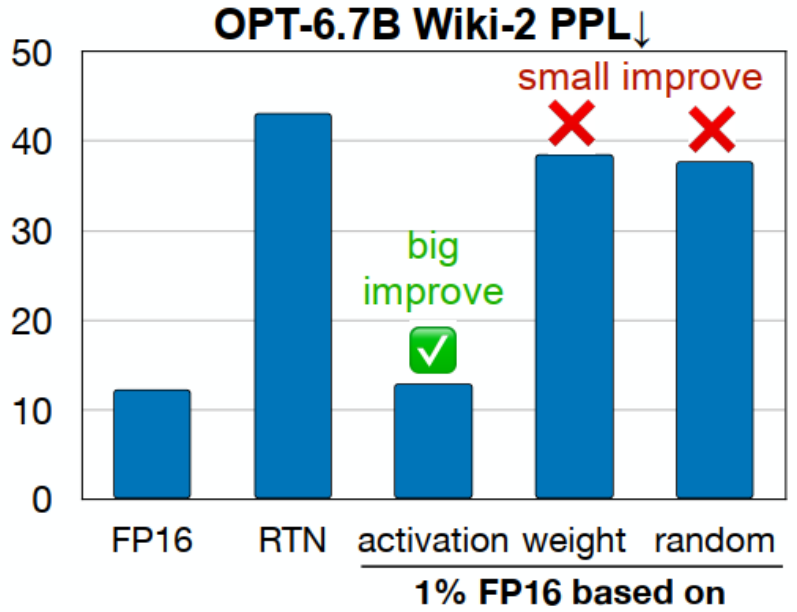
- Observation: Weights are not equally important
  - Keeping only 1% of salient weight channels in FP16 can greatly improve perplexity (43.2 → 13.0)
  - Salient weights are determined by activation distribution, not weight.
  - But the mixed-precision format is not hardware-efficient.



(a) RTN quantization (**PPL 43.2**)



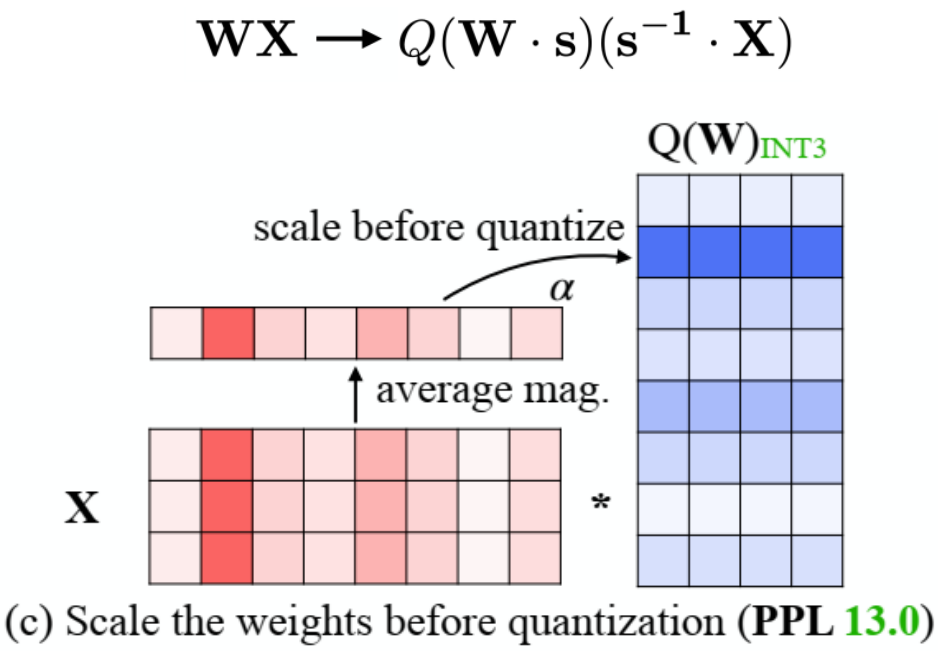
(b) Keep 1% salient weights in FP16 (**PPL 13.0**)



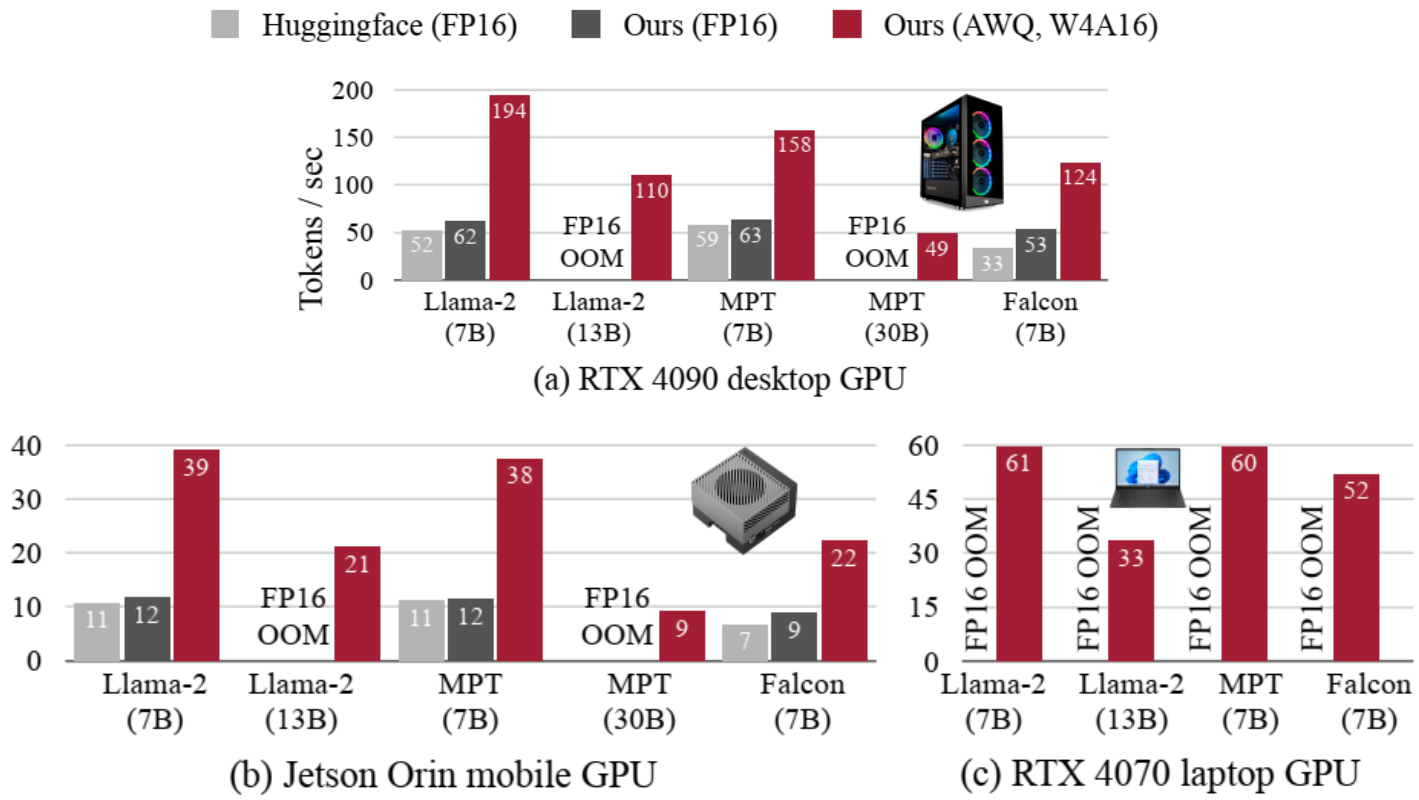
AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration [Lin et al., MLSys 2024]

AWQ

- Protecting salient weights by per-channel scaling
  - Multiplying the salient channels with $s > 1$ reduces its quantization error



(c) Scale the weights before quantization (**PPL 13.0**)



(a) RTX 4090 desktop GPU

(b) Jetson Orin mobile GPU

(c) RTX 4070 laptop GPU

AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration [Lin et al., MLSys 2024]

AWQ

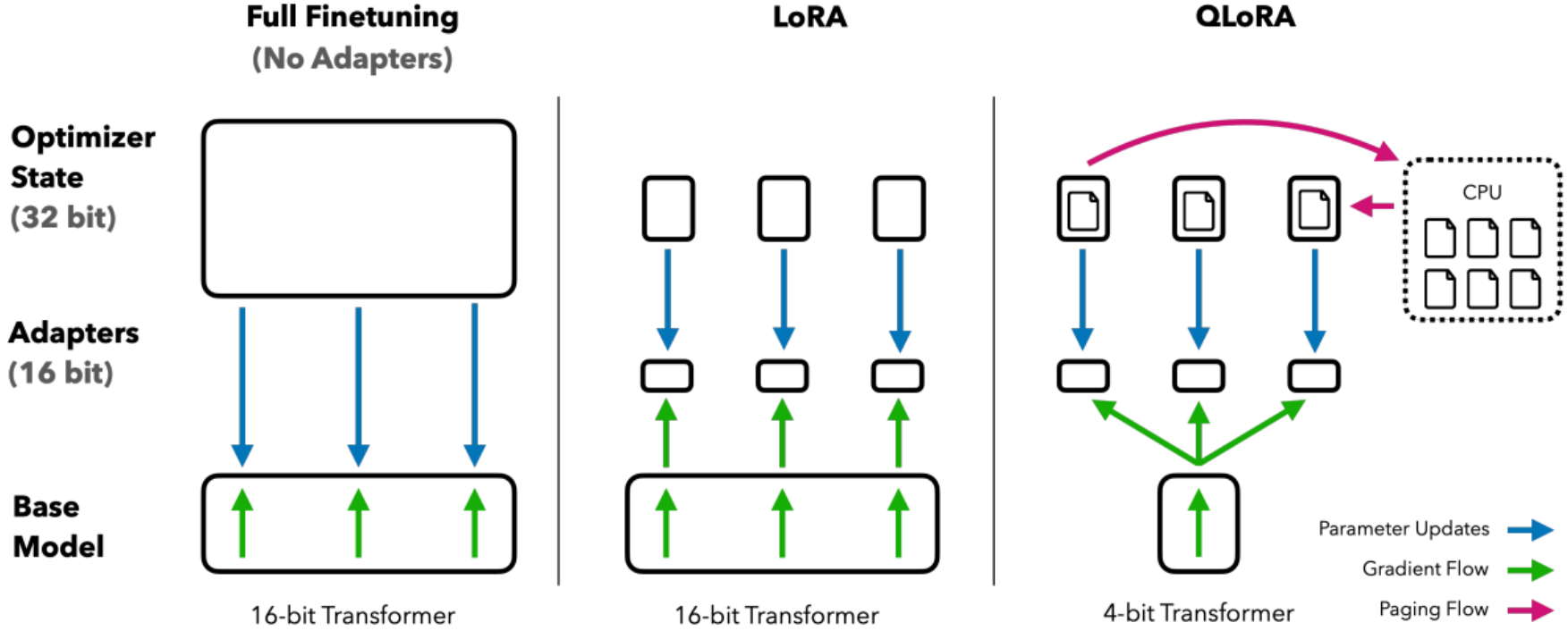
- Better perplexity than GPTQ (w/ and w/o reordering) on LLaMA & Llama-2 models.
  - Reconstruction process of GPTQ leads to an over-fitting issue to the calibration set and may not preserve the generalist abilities of LLMs.

PPL↓		Llama-2			LLaMA			
		7B	13B	70B	7B	13B	30B	65B
FP16	-	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128	RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
	GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
	GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
	AWQ	**6.24**	**5.32**	**3.74**	**6.35**	**5.52**	**4.61**	**3.95**
INT4 g128	RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
	GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
	GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
	AWQ	**5.60**	**4.97**	**3.41**	**5.78**	**5.19**	**4.21**	**3.62**

AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration [Lin et al., MLSys 2024]

QLoRA = LoRA+Quant

- LoRA with quantized base model weights
- NormalFloat (NF4), a new data type for LLM weight quantization
- Double quantization (apply quantization on scaling factors) to further reduce base model size
- Paged Optimizers with CPU offloading



QLoRA: Efficient Finetuning of Quantized LLMs [Dettmers et al., NeurIPS 2023]

NF4 Quantization

- NormalFloat (NF) data type builds on Quantile Quantization
  - Information-theoretically optimal data type
  - Ensures each quantization bin has an equal number of values assigned from the input tensor
- i-th quantized value q_i of the k-bit data type
  - $q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k+1}\right) + Q_X\left(\frac{i+1}{2^k+1}\right)\right)$
  - $Q_X(\cdot)$: quantile function of the standard normal distribution $\sim N(0,1)$

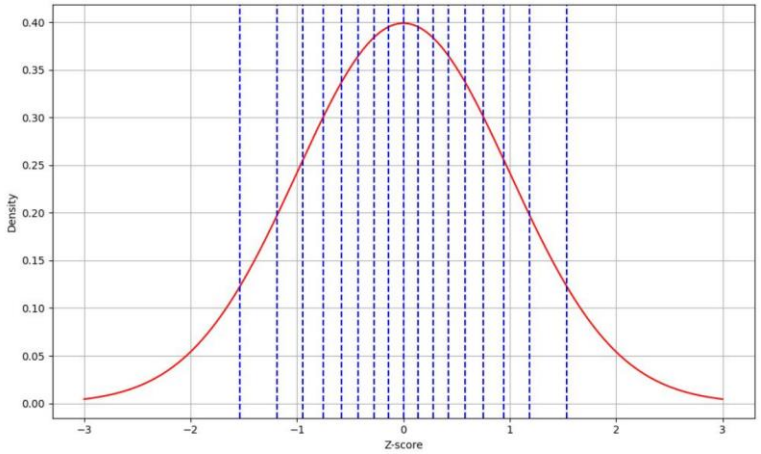


Table 4: Mean 5-shot MMLU test accuracy for LLaMA 7-65B models finetuned with adapters on Alpaca and FLAN v2 for different data types. Overall, NF4 with double quantization (DQ) matches BFloat16 performance, while FP4 is consistently one percentage point behind both.

	Mean 5-shot MMLU Accuracy								
LLaMA Size	7B		13B		33B		65B		Mean
Dataset	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	Alpaca	FLAN v2	
BFloat16	38.4	45.6	47.2	50.6	57.7	60.5	61.8	62.5	53.0
Float4	37.2	44.0	47.3	50.0	55.9	58.5	61.3	63.3	52.2
NFloat4 + DQ	39.0	44.5	47.5	50.7	57.3	59.2	61.8	63.9	53.1

QLoRA: Efficient Finetuning of Quantized LLMs [Dettmers et al., NeurIPS 2023]

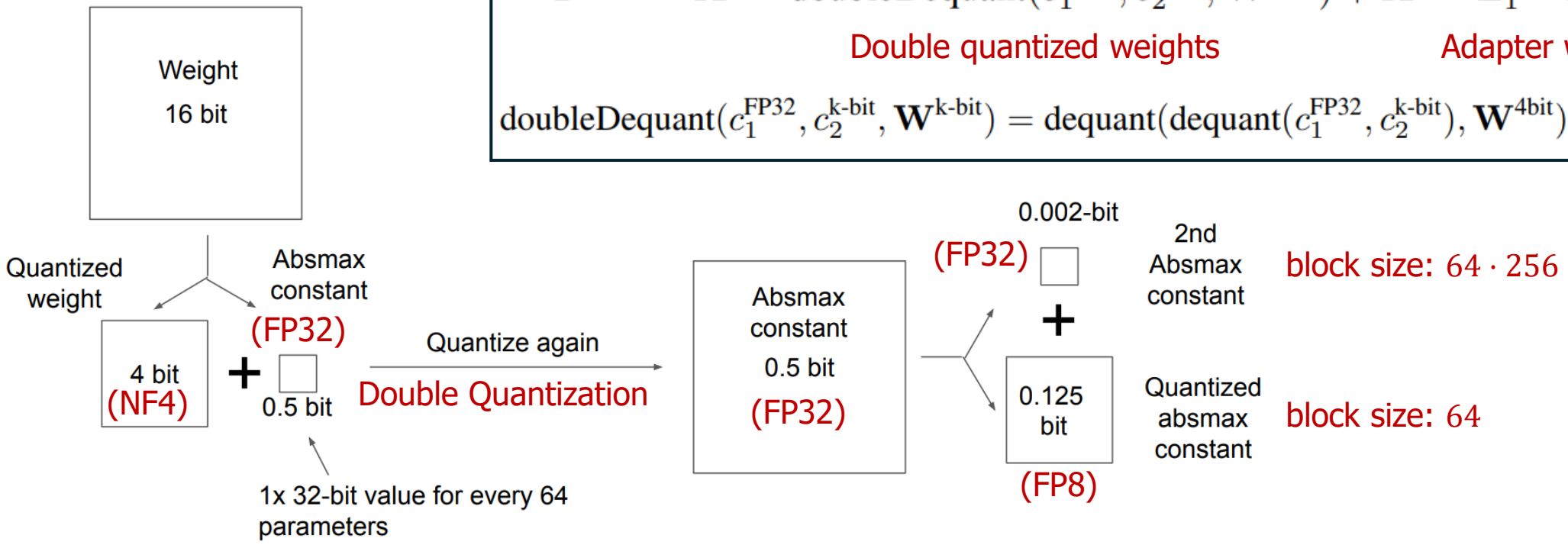
Double Quantization

- Quantize the quantization constants
  - Bits per parameter: $\frac{32}{64} = 0.5$ bits ➔ $\frac{8}{64} + \frac{32}{64 \cdot 256} = 0.127$ bits
- Reduce the memory footprint of quantization constants.
- Compute in BF16

$$\mathbf{Y}^{\text{BF16}} = \mathbf{X}^{\text{BF16}} \text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{NF4}}) + \mathbf{X}^{\text{BF16}} \mathbf{L}_1^{\text{BF16}} \mathbf{L}_2^{\text{BF16}}$$

Double quantized weights — Adapter weights

$$\text{doubleDequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}, \mathbf{W}^{\text{k-bit}}) = \text{dequant}(\text{dequant}(c_1^{\text{FP32}}, c_2^{\text{k-bit}}), \mathbf{W}^{\text{4bit}}) = \mathbf{W}^{\text{BF16}}$$



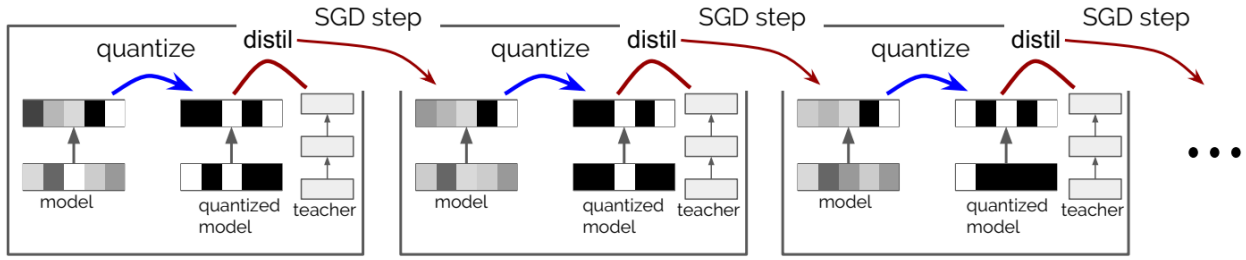
QLoRA: Efficient Finetuning of Quantized LLMs [Dettmers et al., NeurIPS 2023]

Distillation-Assisted Quantization

- The student is shallower than the teacher and quantized.
- To employ distillation loss in the context of a quantized neural network, accumulate the error at each projection step into the gradient for the next step.
- Teacher model: the standard ResNet34 with 87.2-MB parameters
- Student model: 4-bit quantized with only 22-MB parameters.
  - 73.1% top-1 accuracy (better than the 69.75% top-1 accuracy of the standard ResNet18 with 46.8-MB parameters)

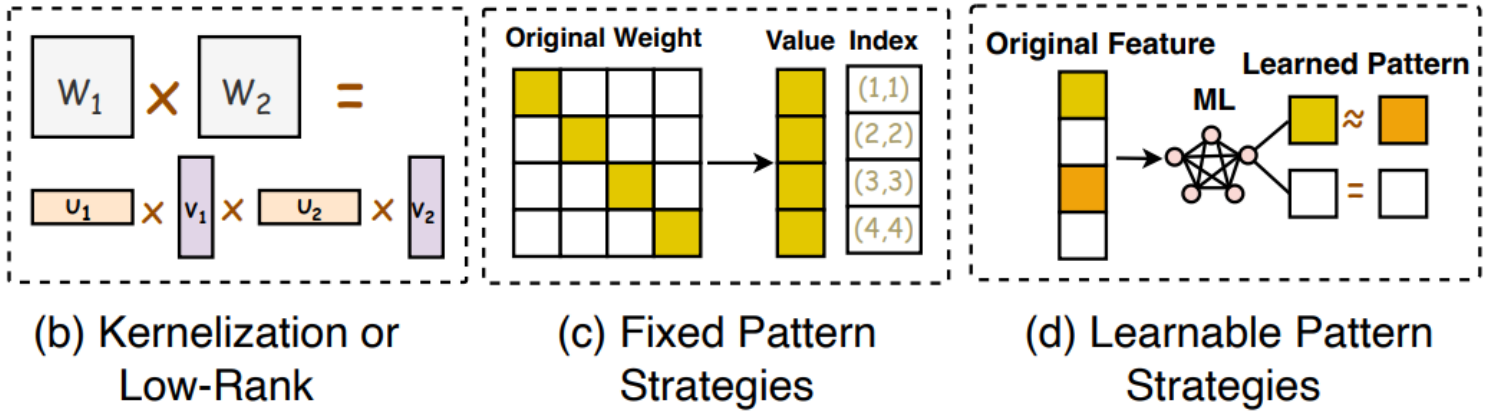
Algorithm 1 Quantized Distillation

procedure QUANTIZED DISTILLATION
  Let w *be the network weights*
loop
  $w^q \leftarrow \text{quant_function}(w, s)$
  Run forward pass and compute distillation loss $l(w^q)$
  Run backward pass and compute $\frac{\partial l(w^q)}{\partial w^q}$
  Update original weights using SGD ***in full precision*** $w = w - \nu \cdot \frac{\partial l(w^q)}{\partial w^q}$
Finally quantize the weights before returning: $w^q \leftarrow \text{quant_function}(w, s)$
return w^q



Model compression via distillation and quantization [ICLR, 2018]
Apprentice: Using Knowledge Distillation Techniques To Improve Low-Precision Network Accuracy [ICLR, 2018]

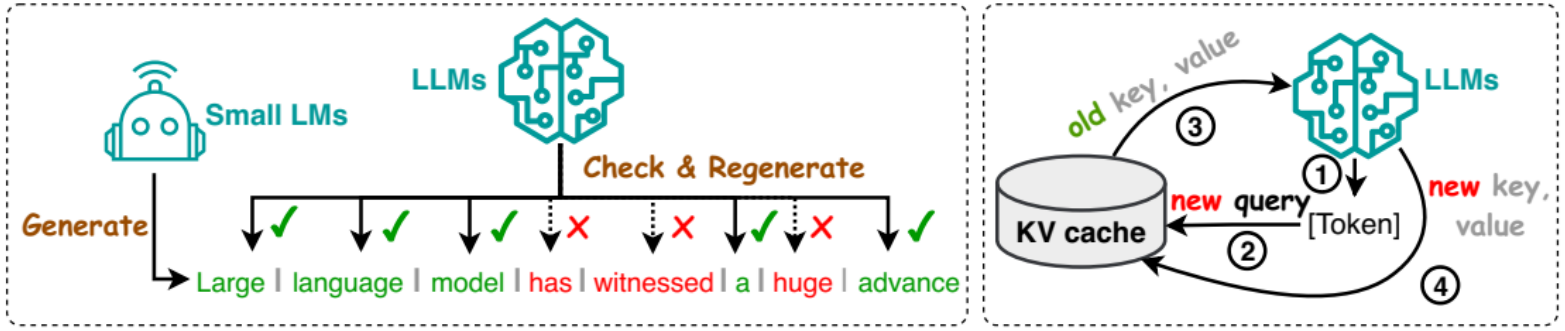
Efficient Transformer



(b) Kernelization or Low-Rank

(c) Fixed Pattern Strategies

(d) Learnable Pattern Strategies



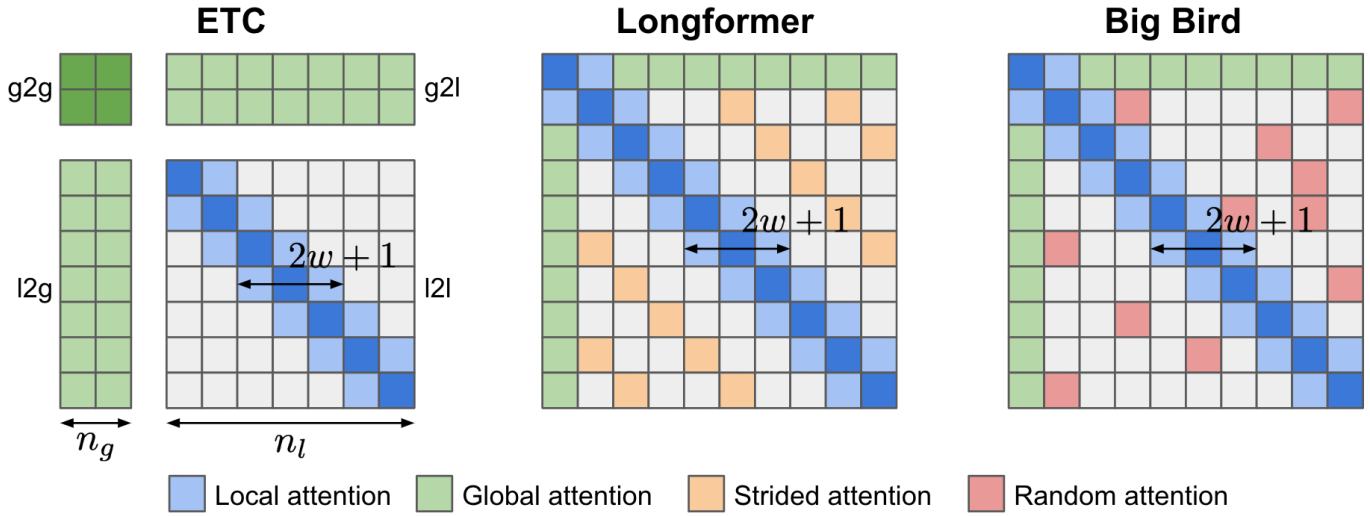
(a) Speculative Decoding

(b) KV-Cache Optimization

Efficient Large Language Models: A Survey [Wan *et al.*, arXiv 2023]

Fixed Sparse Attention

- **Longformer**
  - Local Attention
  - Global Attention
    - global memory tokens that have access to all input sequences.
  - Dilated Sliding Windows
    - Smaller window size at lower layers and larger window sizes at higher layers
- **Big Bird**
  - replaces dilated attention with random attention

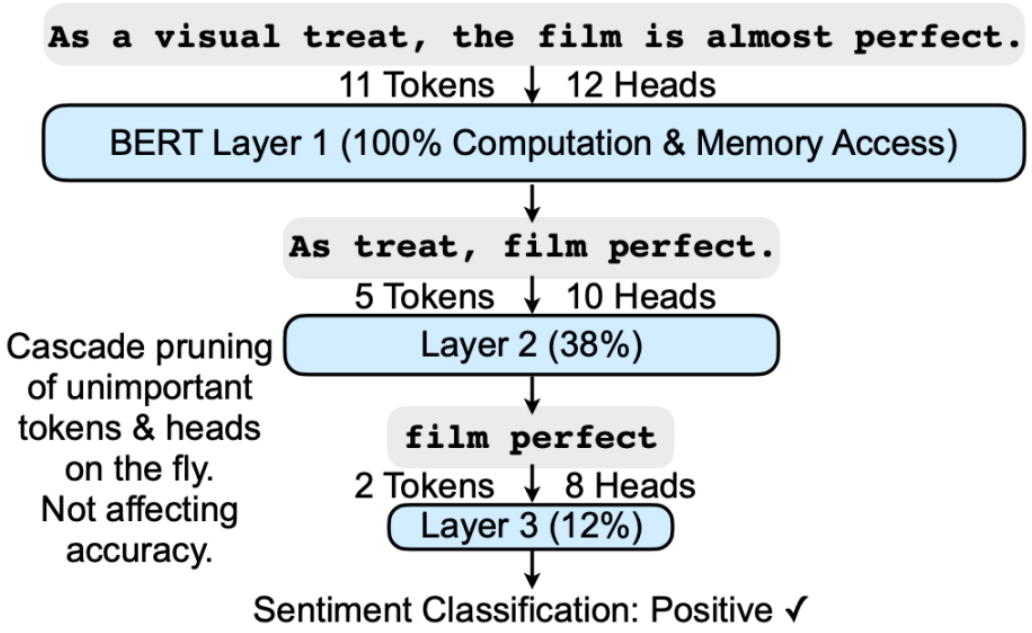


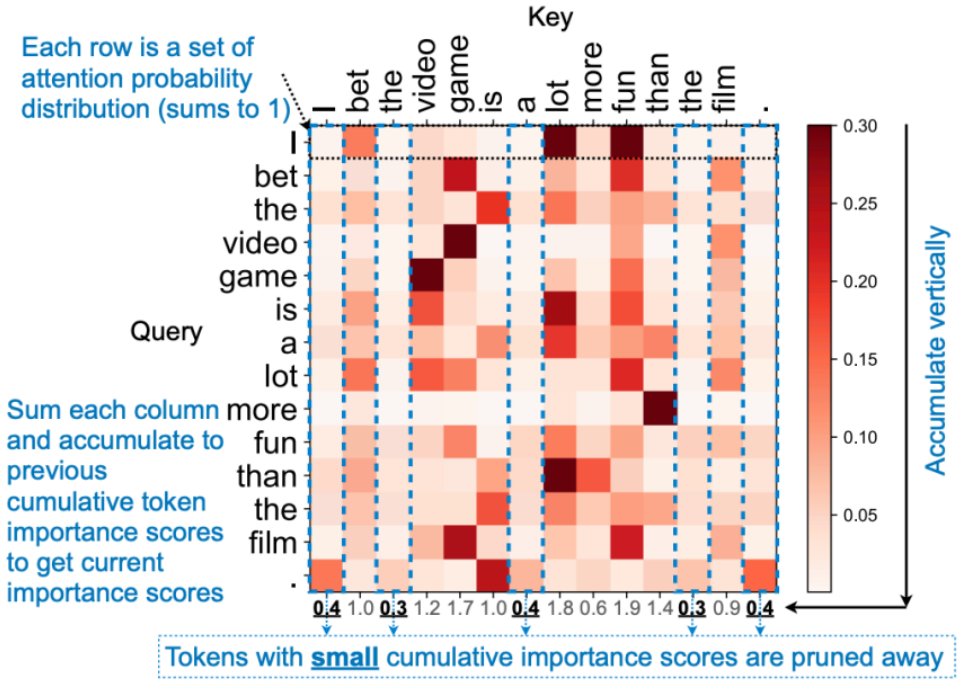
Longformer: The long-document transformer (2020)

Big Bird: Transformers for Longer Sequences (NeurIPS 2020)

Dynamic Sparsity - SpAtten

- Token pruning & head pruning
- Cascade pruning of unimportant tokens and heads.
- Tokens with a small cumulative attention are pruned away.
- Value pruning: don't fetch V if QK is small.
- Progressive quantization: low precision first, if not confident => high precision.



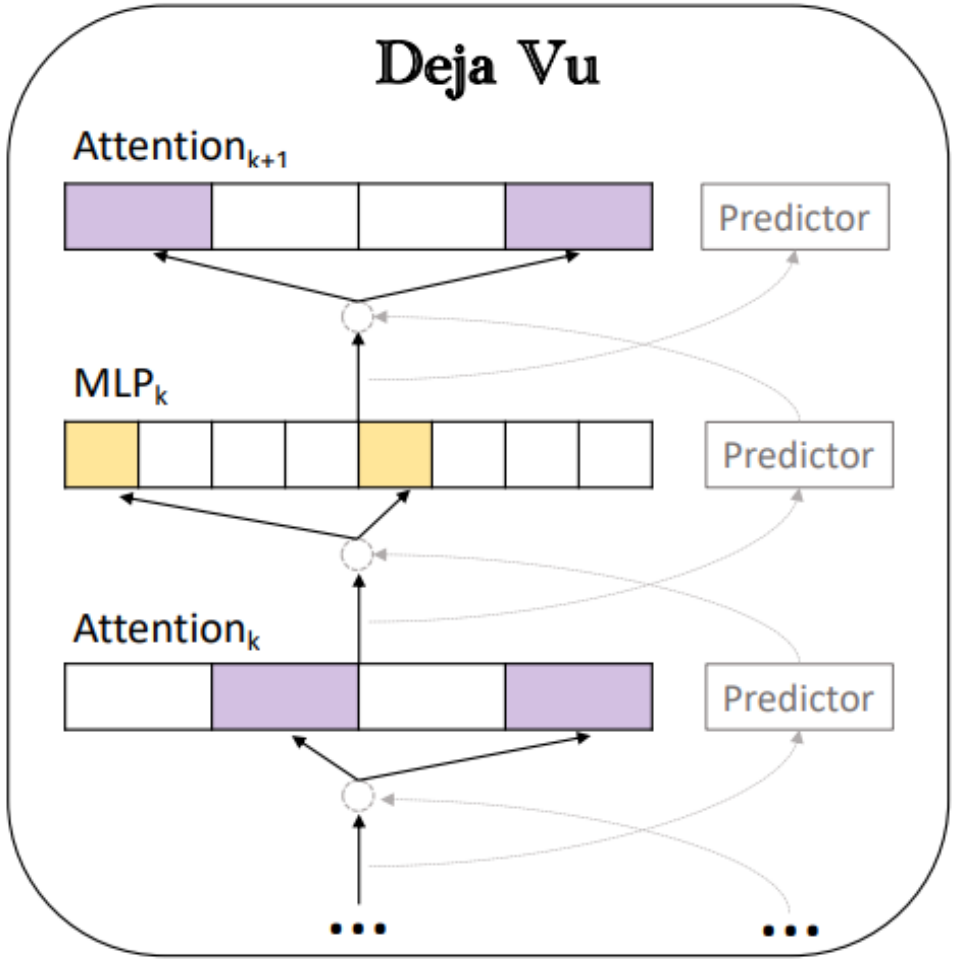


SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning (HPCA'21)

Deja Vu

- Contextual Sparsity Prediction
  - Training additional sparsity predictor for MLP and Attention.
  - Predict the sparsity of MLP inner dimensions and Attention-heads

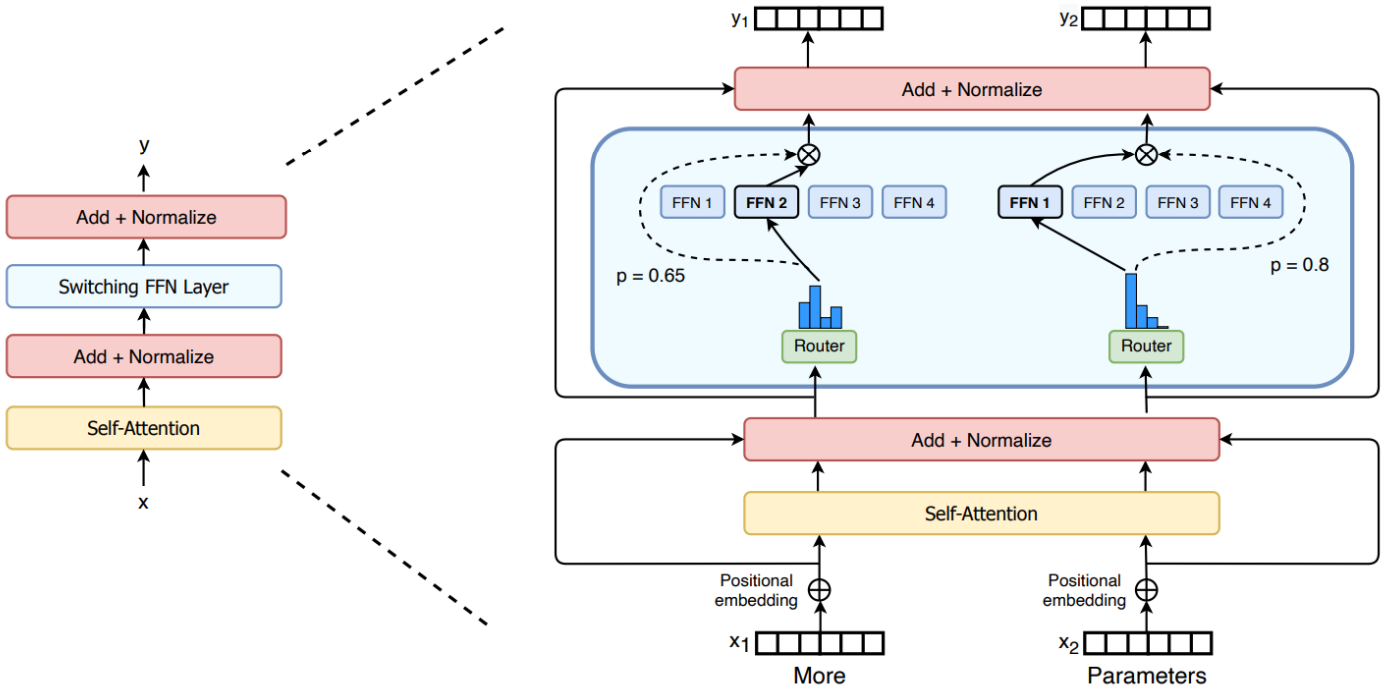
- Asynchronous Execution
  - Leverage slowly evolving embedding phenomenon
  - Look-ahead sparse prediction
  - Parallel execution of the sparse prediction with the computation by using previous block's outputs.



Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time [Liu et al., ICML 2023]

Conditional Computation

- Switch Transformers
  - Mixture of Experts (MoE) to select different parameters for each incoming example (MoE routing)
  - Smaller FFN parameters

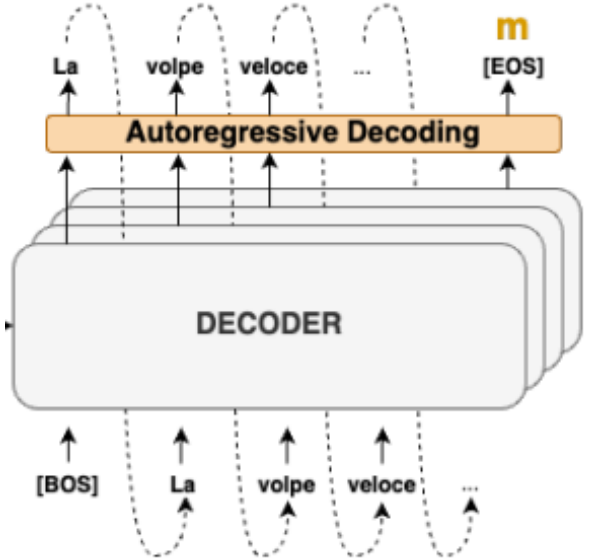


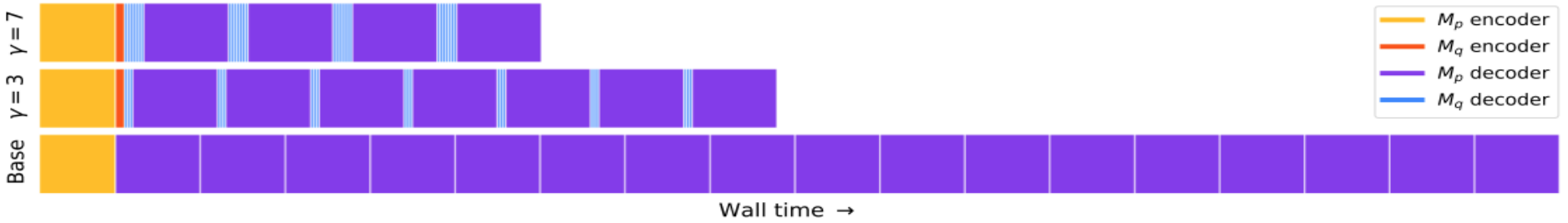
The dense feed forward network (FFN) layer is replaced with a sparse Switch FFN layer (light blue).
The layer operates independently on the tokens in the sequence.

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity [JMLR 2022]

Speculative Decoding

- **Motivations**
  - Traditional auto-regressive models need sequential calls for each token, causing memory limits.
  - Some inference steps are harder, and some are easier
- **Speculative Decoding**
  - M_q: small, efficient, and approximation model
    - Called sequentially γ times to generate tokens
  - M_p : large target model
    - Evaluate the guesses and probabilities from M_q in parallel
    - Accept those that can lead to an identical distribution
    - Sample an additional token to fix the rejected one, or to add an additional one if all are accepted

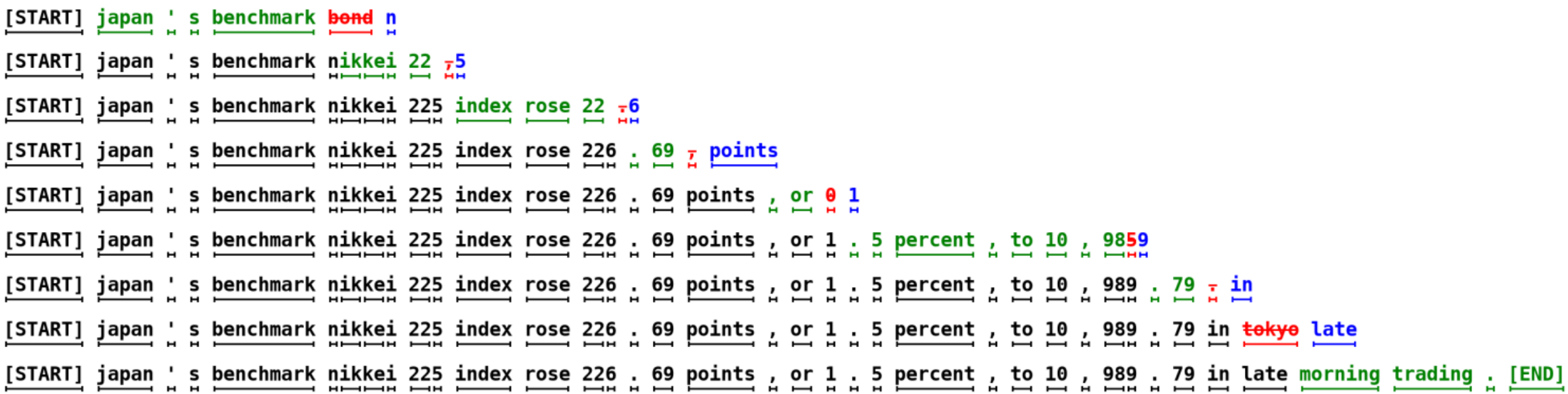




Fast Inference from Transformers via Speculative Decoding [Leviathan et al., ICML 2023]

Speculative Decoding

- The green tokens are the suggestions made by the small model
- The red tokens are the rejected suggestions by target model.
- The blue tokens are correction by target model.
- $\gamma = 5$



Fast Inference from Transformers via Speculative Decoding [Leviathan et al., ICML 2023]

Speculative Decoding

- α: The capability of a smaller model compared to the target model.
- Temp: sampling temperature
- By speculative inference, up to 3.4X speed up is achieved.

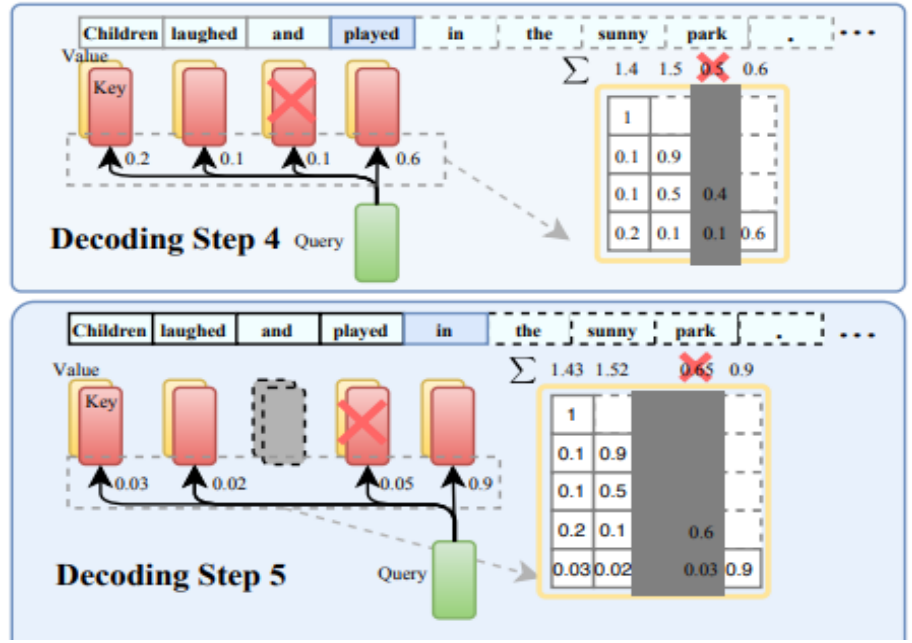
Table 2. Empirical results for speeding up inference from a T5-XXL 11B model.

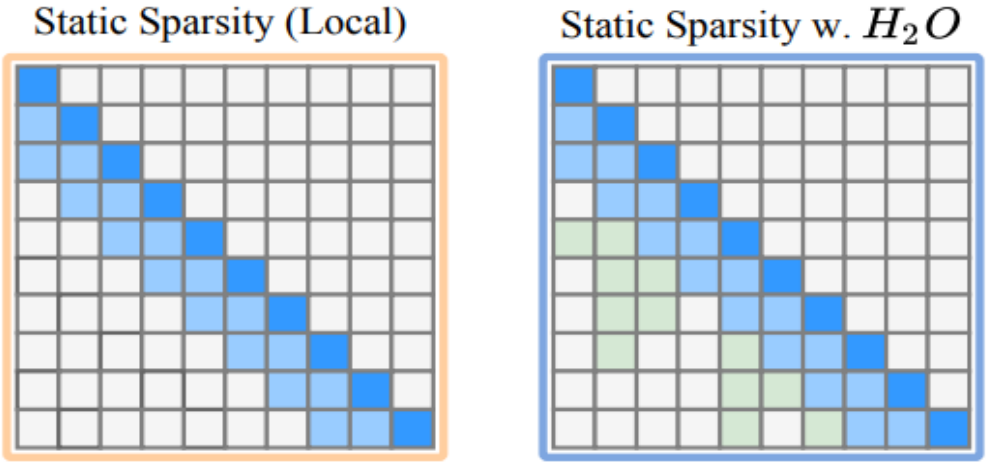
TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	**3.4X**
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	**2.6X**
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	**3.1X**
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	**2.3X**
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

Fast Inference from Transformers via Speculative Decoding [Leviathan et al., ICML 2023]

H_2O

- Only 5% of the KV cache is sufficient for decoding the same output token at each generation step
- retains only the H2 and the recent KV embeddings in the cache
- How to select necessary tokens?
  - A token's importance in attention operations is determined by large attention scores (softmax(Q*K)).
- KV cache entries with low accumulative attention scores are removed.

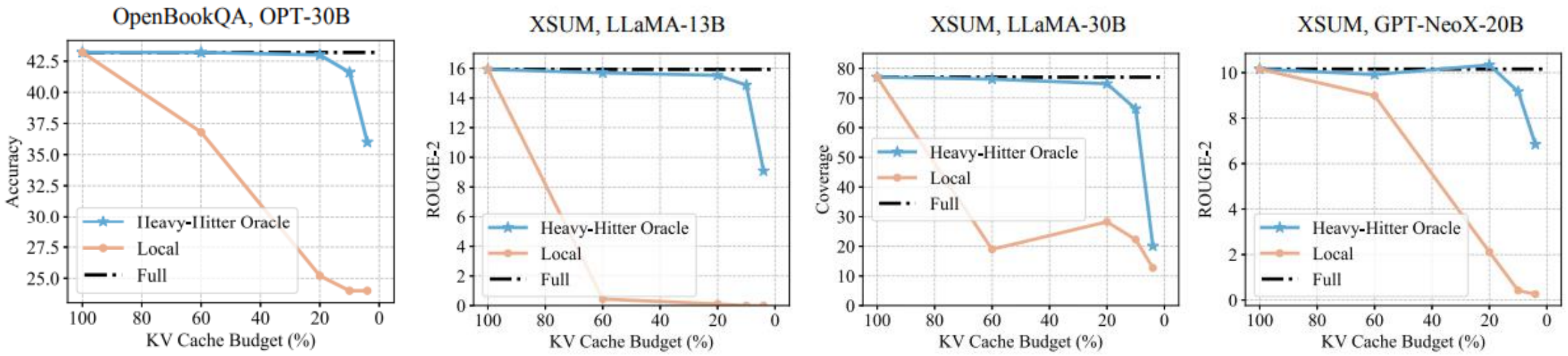




H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models [Zhang et al., NeruIPS 2024]

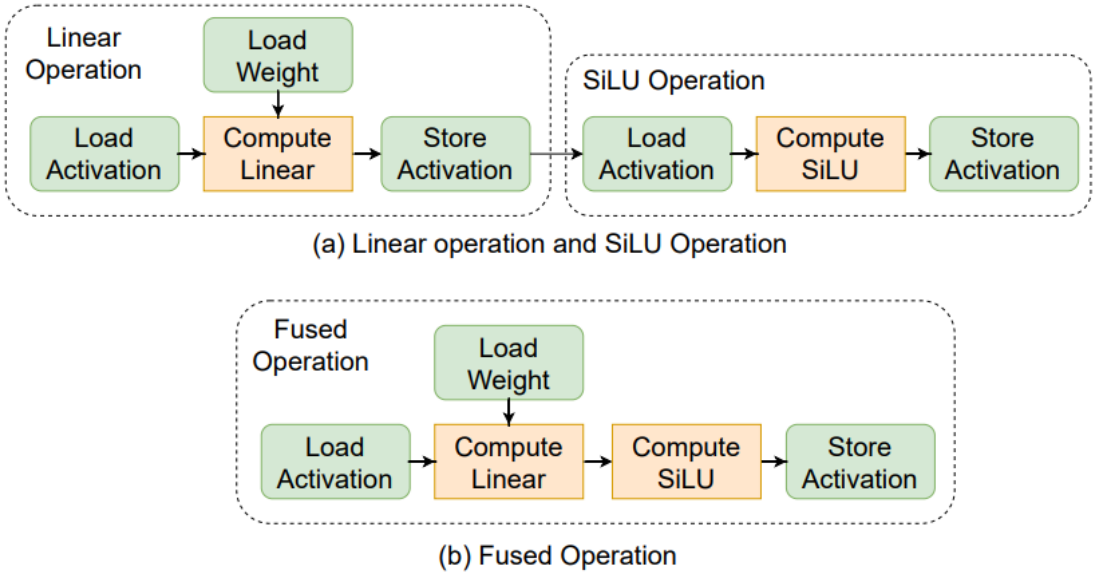
H_2O

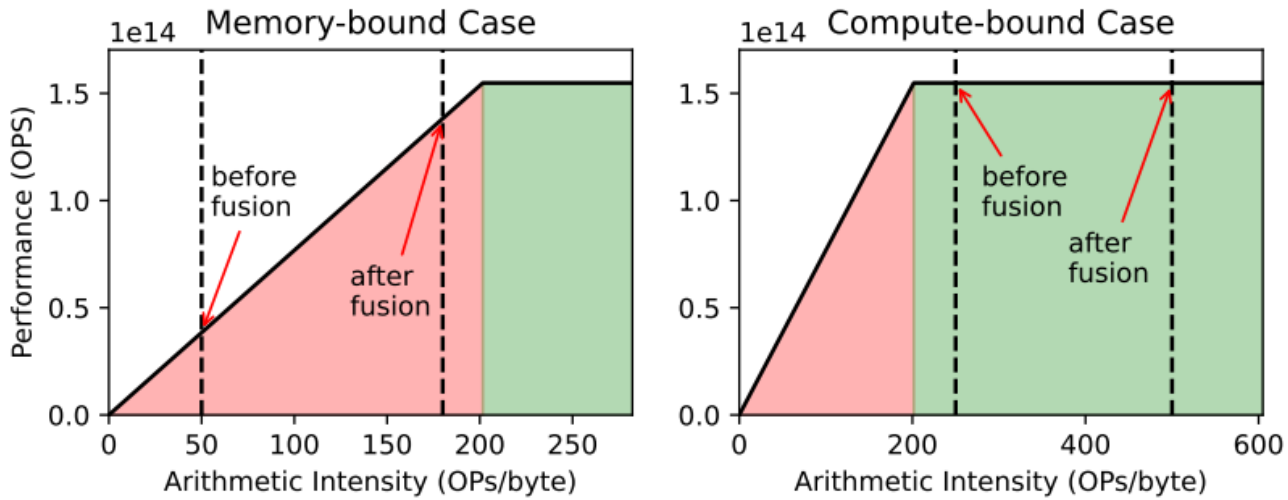
- Significant improvements against the "Local" strategy at different KV cache budgets



H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models [Zhang et al., NeruIPS 2024]

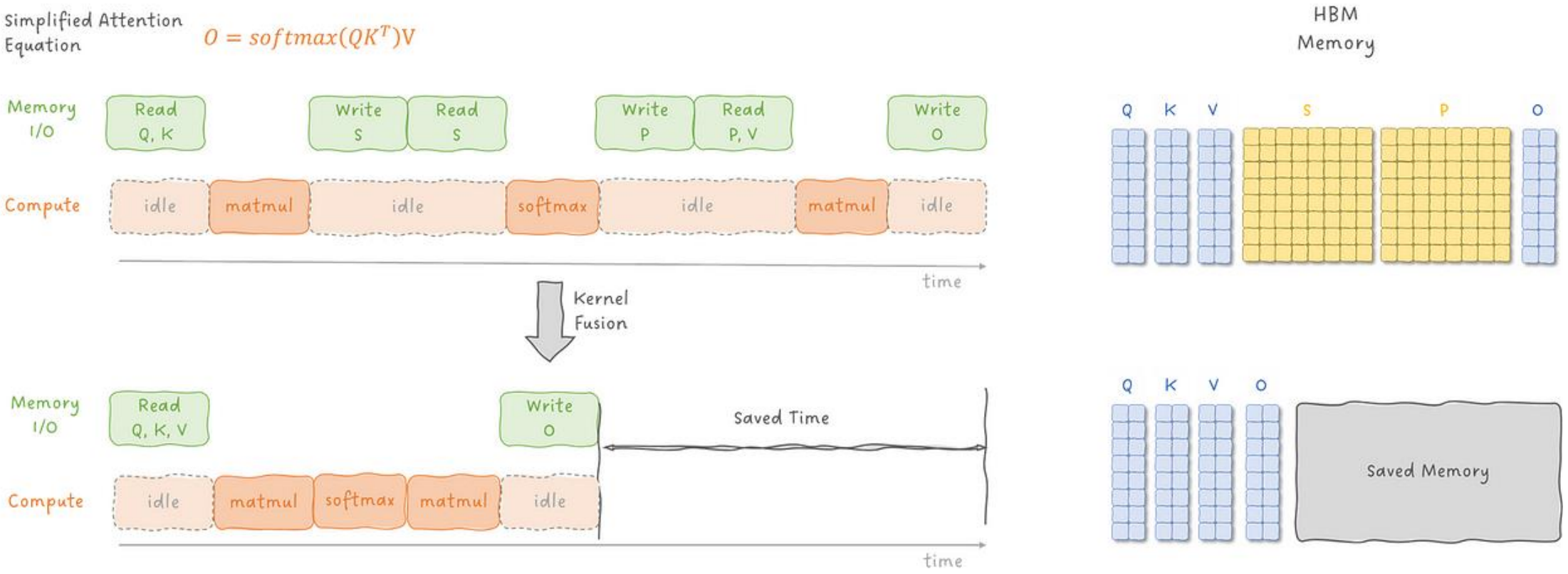
Operator Fusion





LLM Inference Unveiled: Survey and Roofline Model Insights [ArXiv 2024]

Operator Fusion



FlashAttention

- Avoid reading and writing the attention matrix to and from GPU global memory with fused kernel
- Computing the row-wise softmax need to access to whole input
- Softmax reduction: Tiling and Recomputation

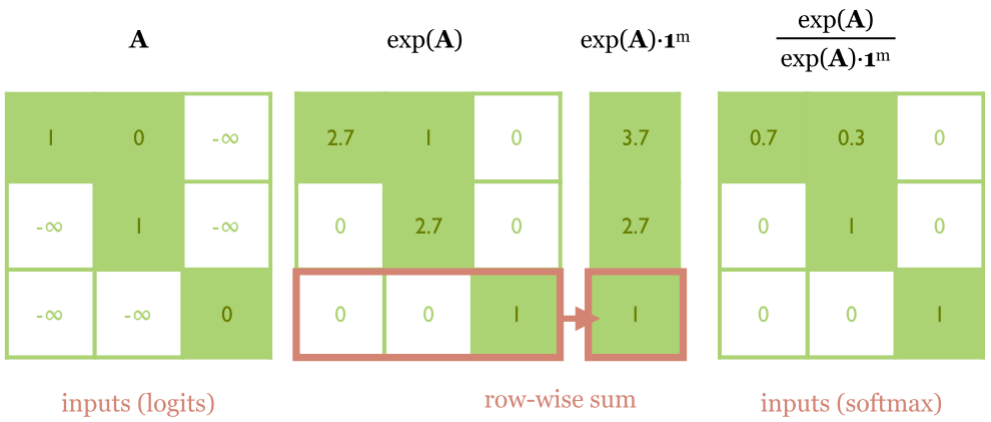
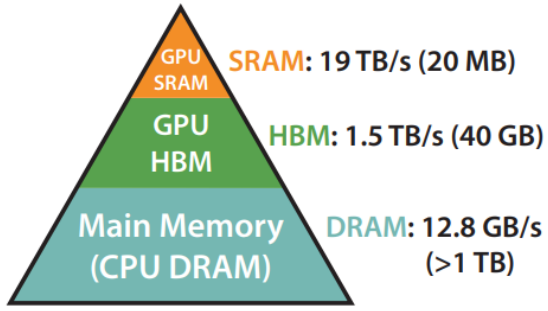
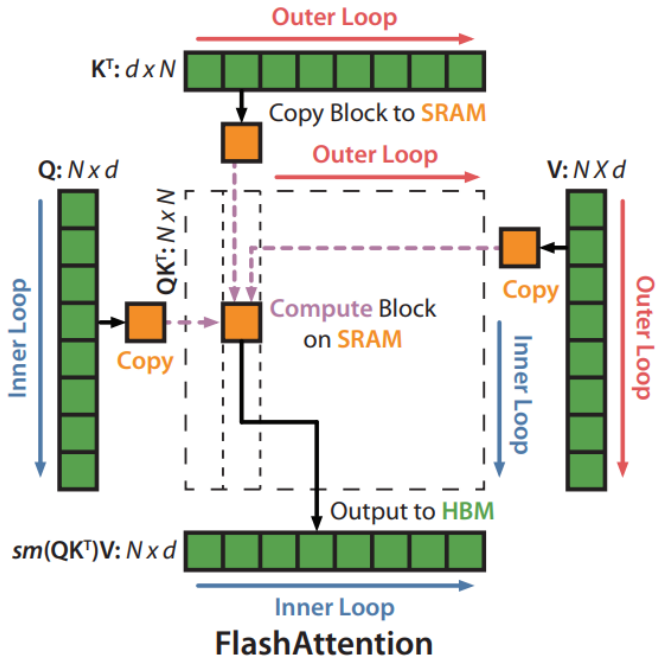


Image from https://peterbloem.nl/blog/stable-softmax





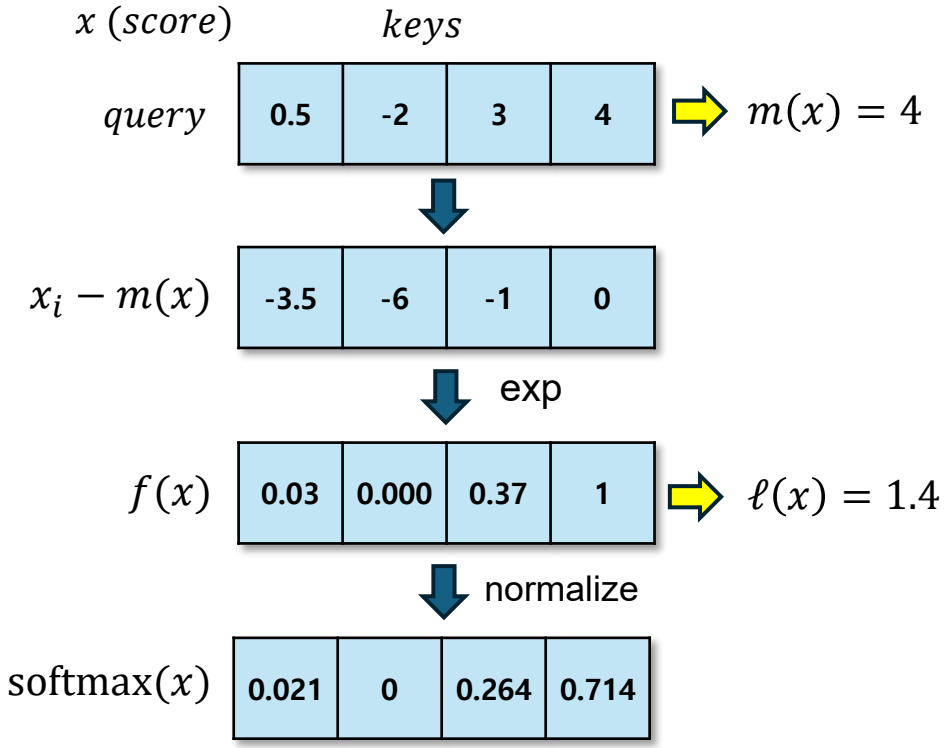
FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness [NeurIPS'22]

FlashAttention

- Standard Softmax vs Tiled Softmax

$$m(x) := \max_i x_i, \quad f(x) := \begin{bmatrix} e^{x_1 - m(x)} & \dots & e^{x_B - m(x)} \end{bmatrix},$$

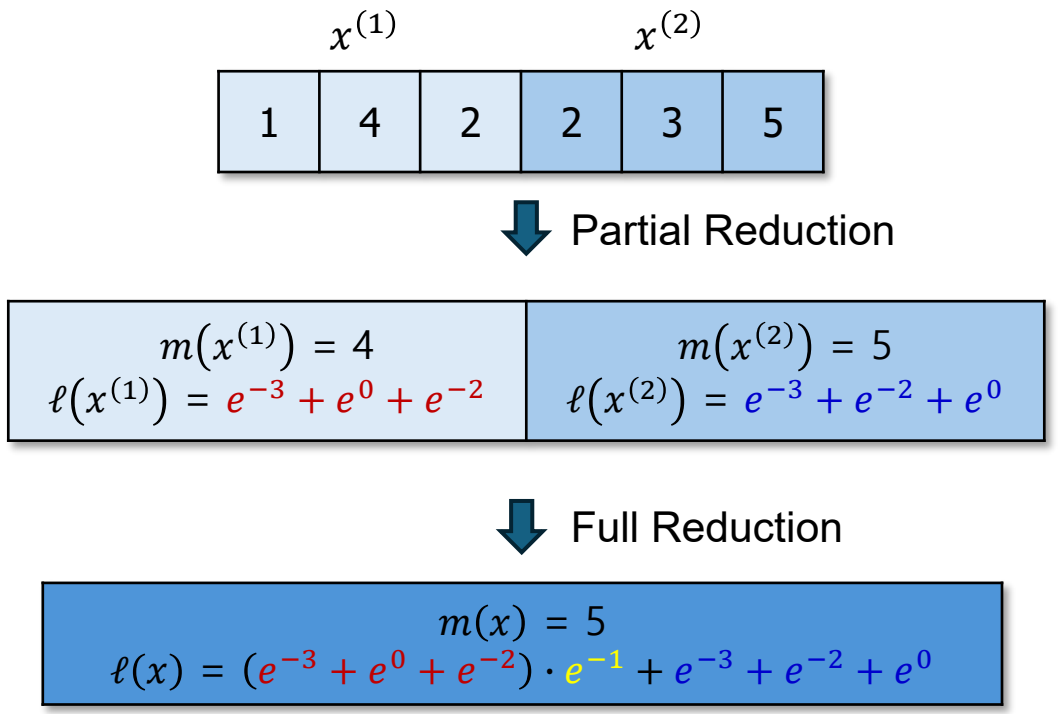
$$\ell(x) := \sum_i f(x)_i, \quad \text{softmax}(x) := \frac{f(x)}{\ell(x)}$$



$$m(x) = m(\begin{bmatrix} x^{(1)} & x^{(2)} \end{bmatrix}) = \max(m(x^{(1)}), m(x^{(2)})),$$

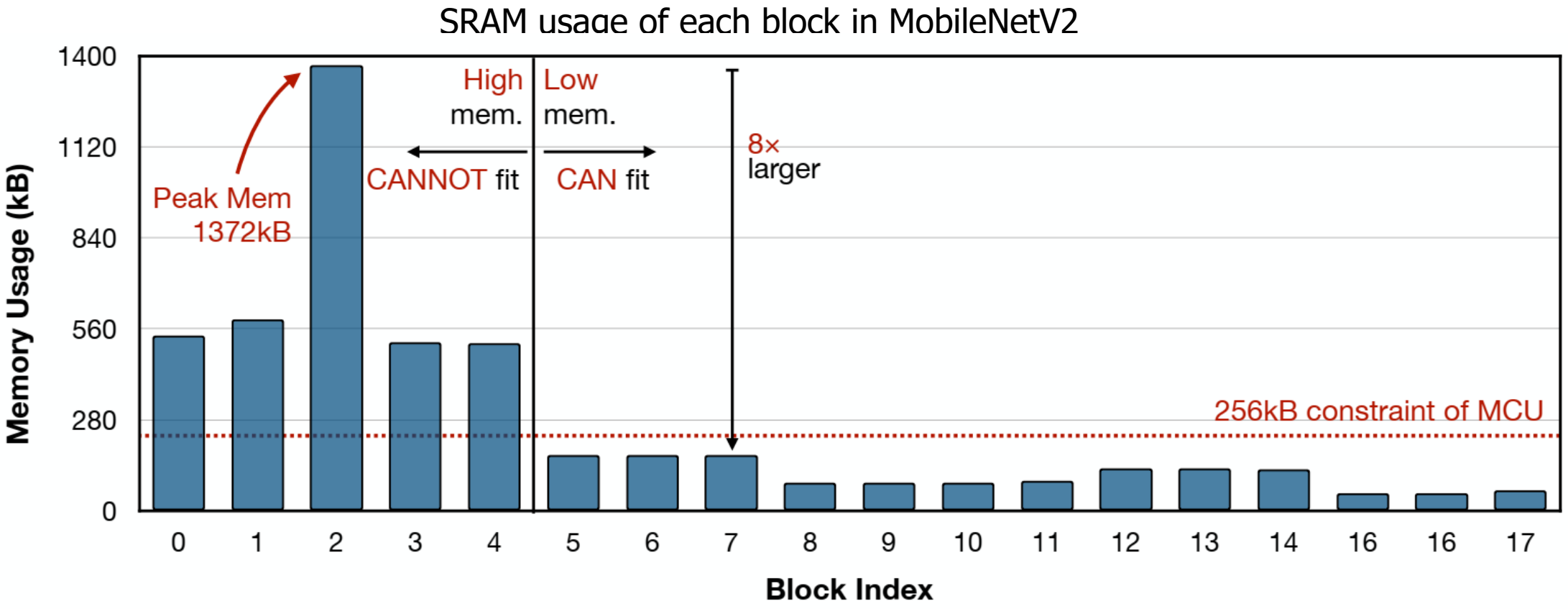
$$f(x) = \begin{bmatrix} e^{m(x^{(1)}) - m(x)} f(x^{(1)}) & e^{m(x^{(2)}) - m(x)} f(x^{(2)}) \end{bmatrix}$$

$$\ell(x) = \ell(\begin{bmatrix} x^{(1)} & x^{(2)} \end{bmatrix}) = e^{m(x^{(1)}) - m(x)} \ell(x^{(1)}) + e^{m(x^{(2)}) - m(x)} \ell(x^{(2)})$$



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness [NeurIPS'22]

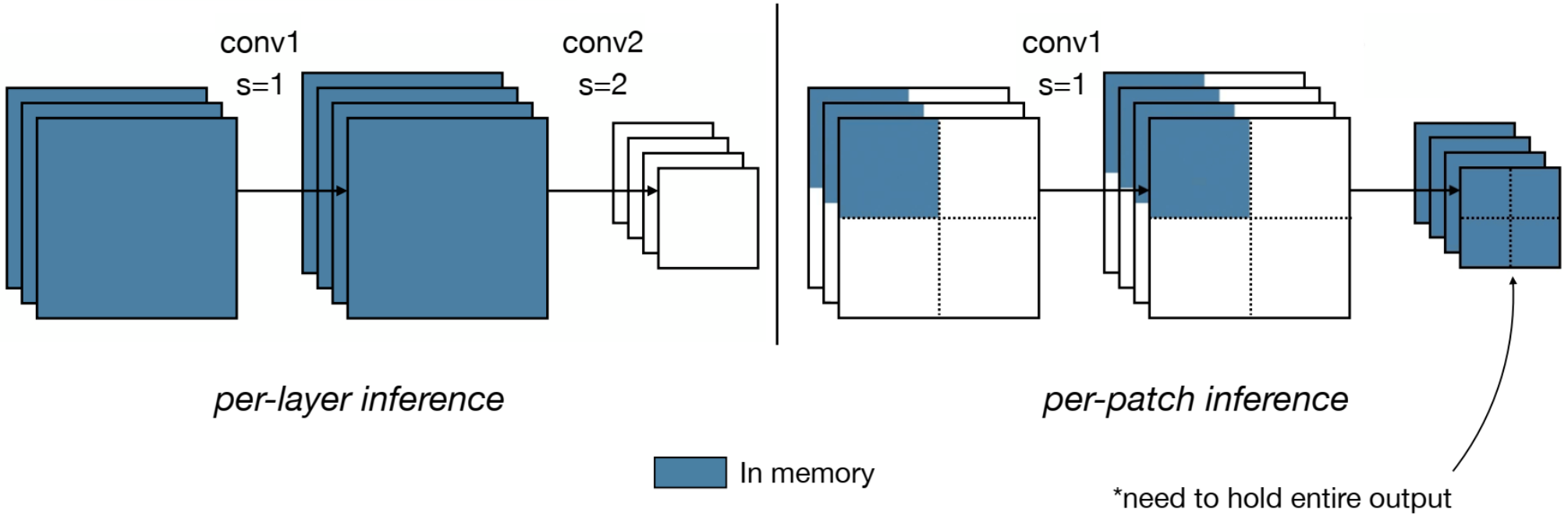
Patch-based Inference



MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning [Lin *et al.*, NeurIPS 2021]

Patch-based Inference

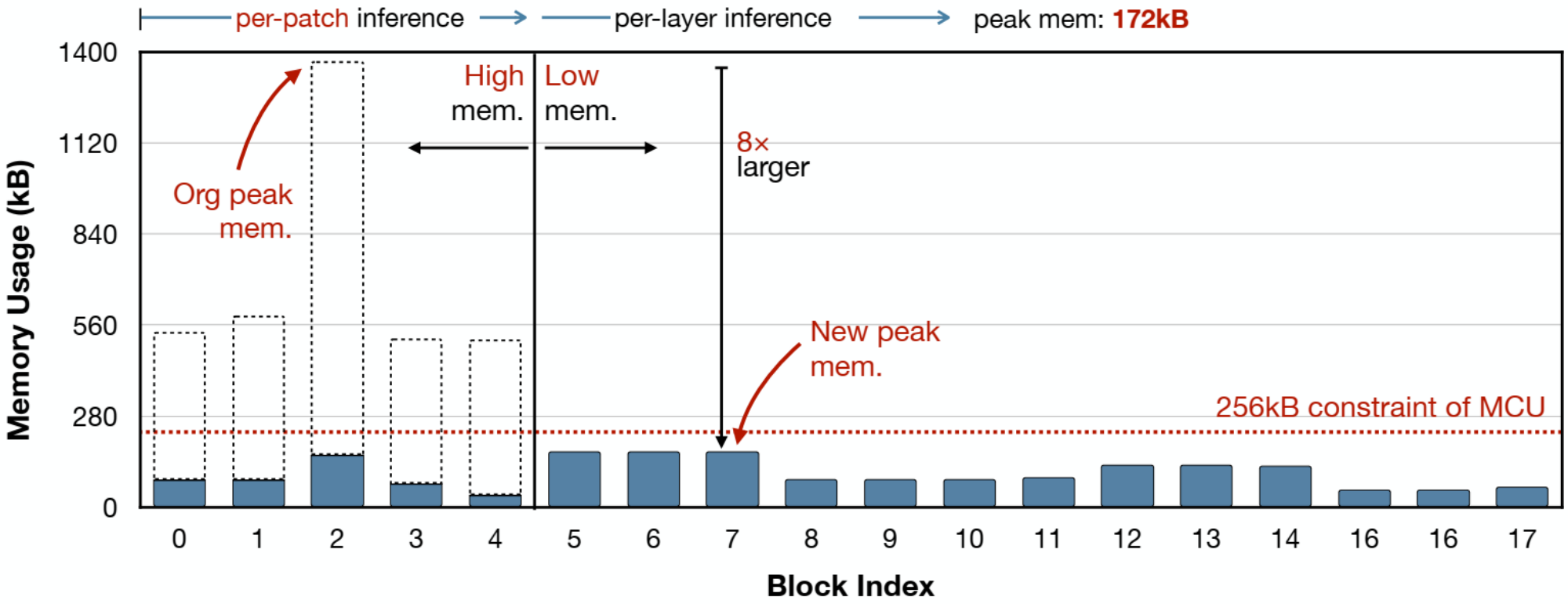
- Saving Memory with Patch-based Inference
  - Break the memory bottleneck with patch-based inference



MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning [Lin *et al.*, NeurIPS 2021]

Patch-based Inference

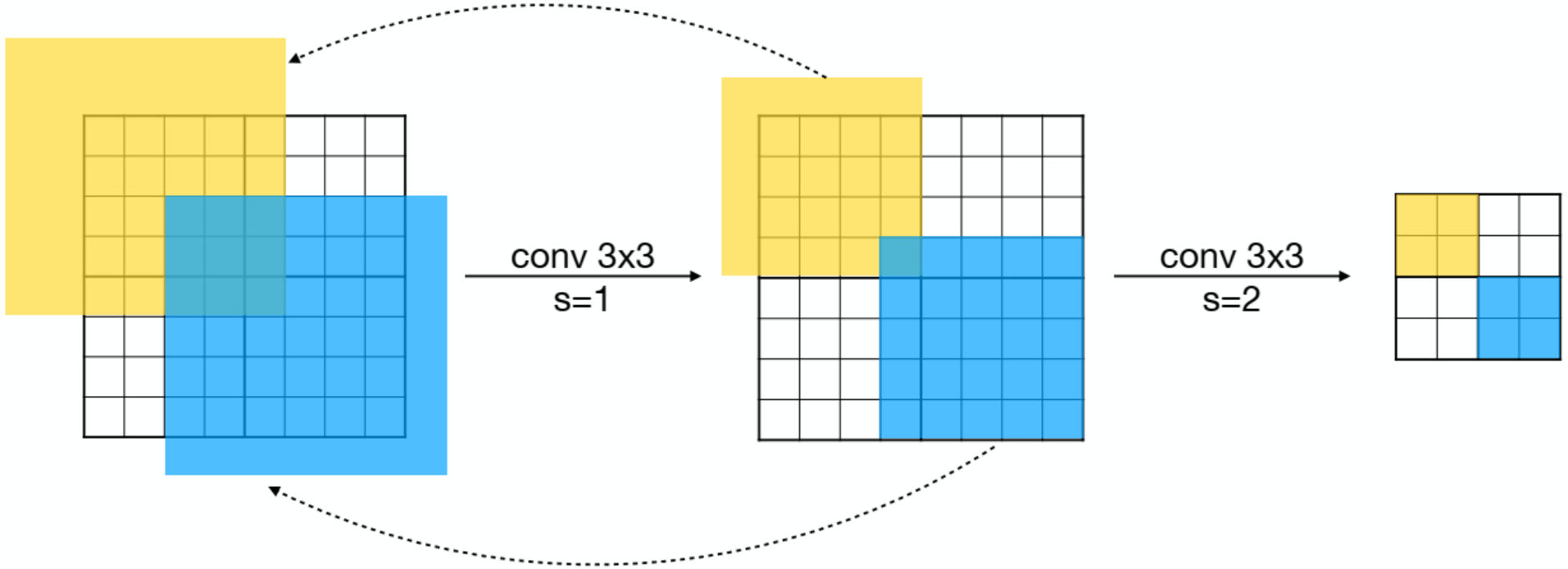
- Saving Memory with Patch-based Inference
  - Memory saving for MobileNetV2



MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning [Lin *et al.*, NeurIPS 2021]

Patch-based Inference

- Problem: halo leads to repeated computation



MCUNetV2: Memory-Efficient Patch-based Inference for Tiny Deep Learning [Lin *et al.*, NeurIPS 2021]